项目4 基于lora微调LLM的多语言电商评论摘要生成实践

一、基础知识

> 本章作为背景知识，不涉及代码，只涉及到知识，面试中问到了，讲讲自己的了解即可。

本章首先介绍大语言模型（Large Language Models， LLM）的训练流程与模型结构，随后重点讲解如何引导模型生成符合需求的输出，包括推理阶段控制、模型微调与 Prompt 工程等方法。掌握这些技术，有助于我们更加高效、稳定地利用 LLM 解决实际问题。

1.1、LLM训练步骤

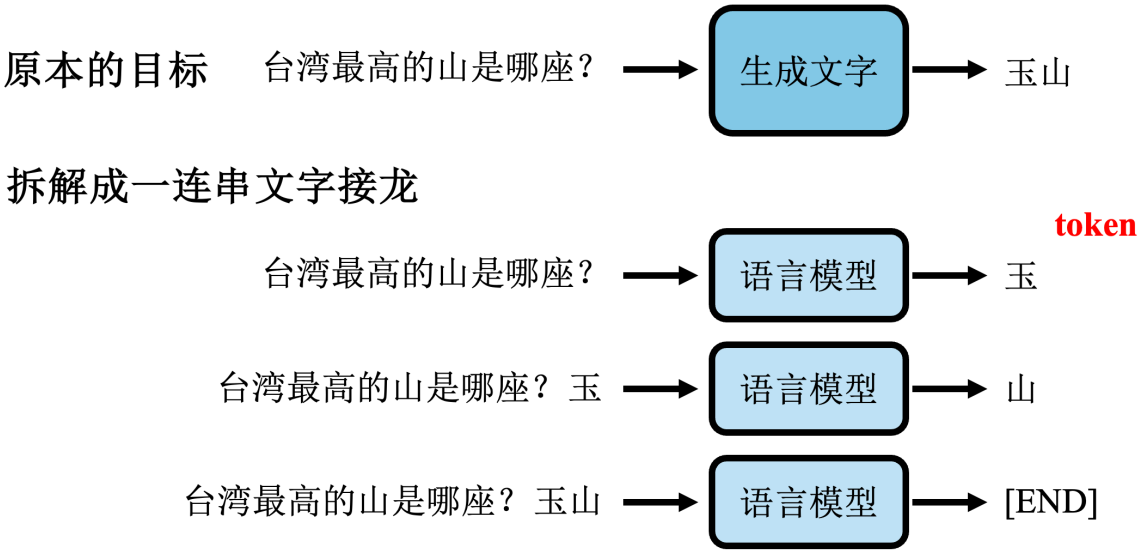
> 了解有哪些步骤即可。

1.1.1、背景知识

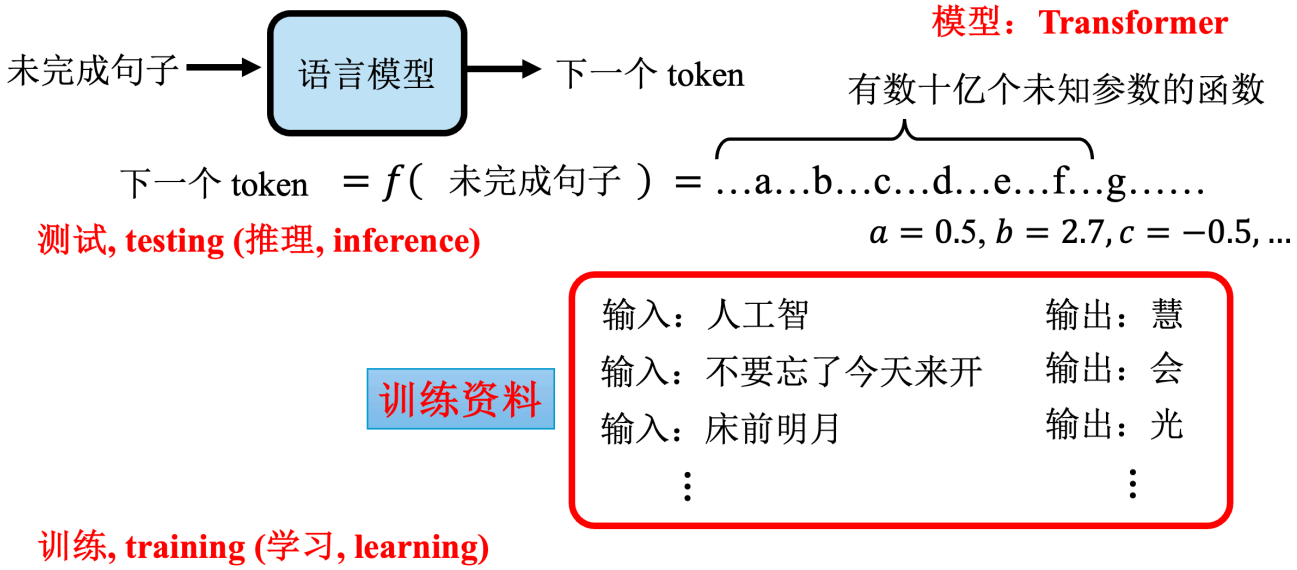
当前主流大语言模型大多采用 Decoder-only 架构。在这种结构中，所有 token 会同时输入到每一层 Transformer，但通过因果掩码限制注意力方向，使每个 token 只能关注自身及之前的 token，从而实现自回归建模。虽然前面的 token 无法直接看到后续 token，但后续 token 会逐层整合前文信息，整体形成从左到右的上下文建模过程。

在生成阶段，给定输入序列如 [人] [工] [智能] [是] [什么] [?] [eos]，序列末尾 token 的表示向量会综合整句话的语义信息，并通过全连接层映射到词表空间，再经过 softmax 得到下一个 token 的概率分布。最后，根据该概率分布选择一个词（如取最大概率或按采样策略），作为模型生成的下一个 token，从而完成 Next Token 预测。

知识背景，文字接龙



背景知识，机器怎么学习会做文字接龙？



为了让LLM更好的学会文字接龙，一般通过三个阶段的训练，分别为预训练阶段、指令微调阶段和对齐优化阶段，但每个阶段本质是都在学文字接龙，只是训练资料不同。

- 预训练：教模型“读书识字”
- 指令微调：教模型“听懂问题并作答”

- 对齐优化（RLHF）：教模型“什么样的回答是好回答”

第一阶段
自我学习，积累功力

第二阶段
名师指点，发挥潜力

第三阶段
参与战争，打磨技巧

所有的阶段都在学文字接龙，只是训练资料不同

1.1.2、预训练阶段

预训练的目标是让模型掌握语言结构、语义关系、常识知识与基本推理能力。

训练任务通常为预测下一个词元。模型在给定前文的条件下，学习对下一个词元的概率分布进行建模。通过大规模语料的反复训练，模型逐步形成对语言的整体理解能力。

第一阶段
自我学习，积累功力

第二阶段
名师指点，发挥潜力

第三阶段
参与战争，打磨技巧

需要多少文字才能够学会文字接龙？

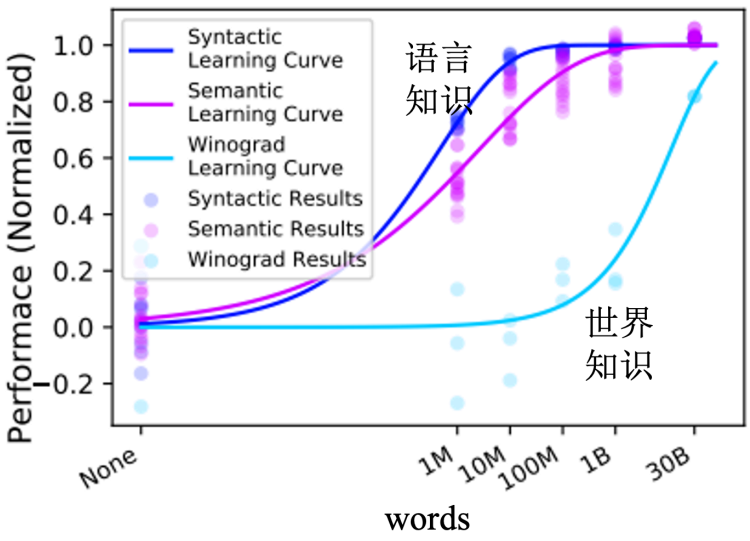
https://arxiv.org/abs/2011.04946

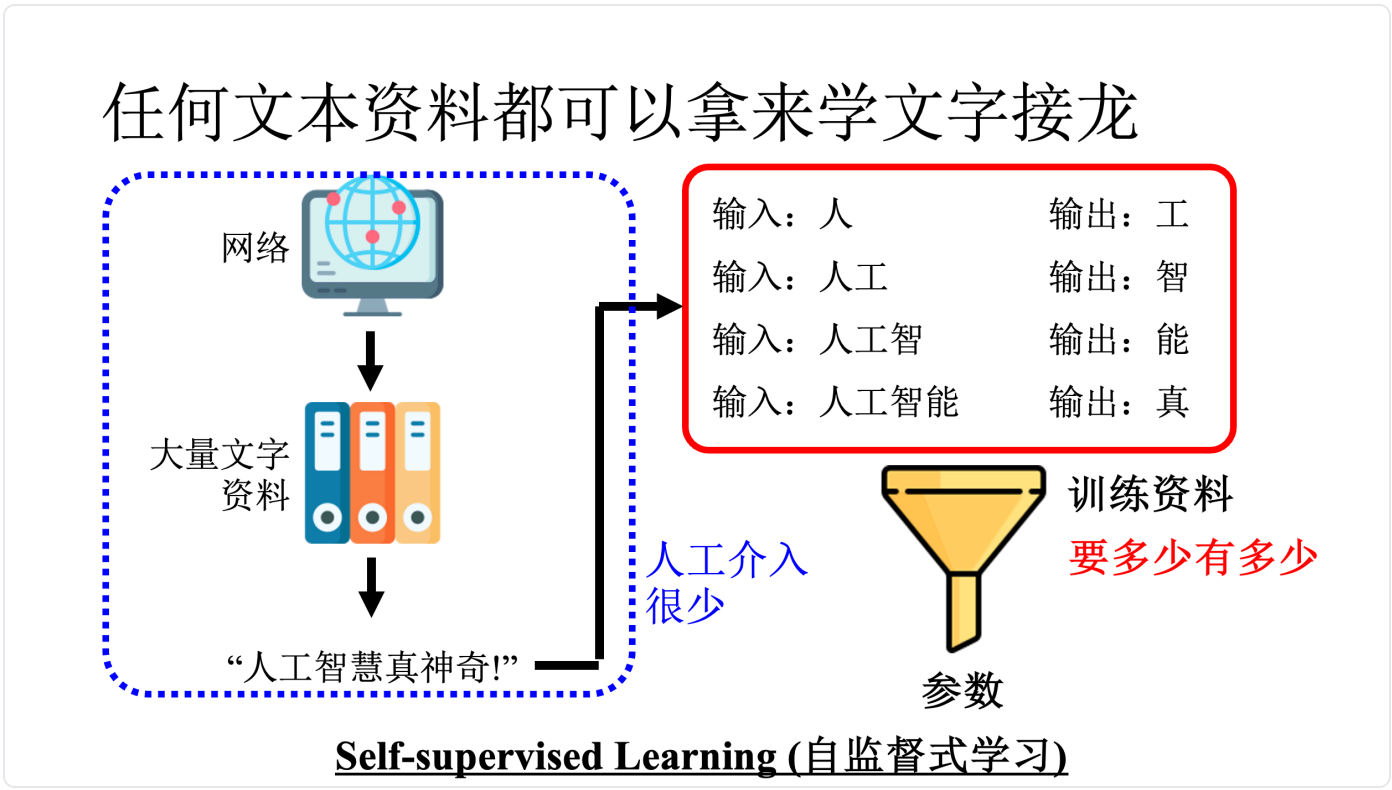
语言知识

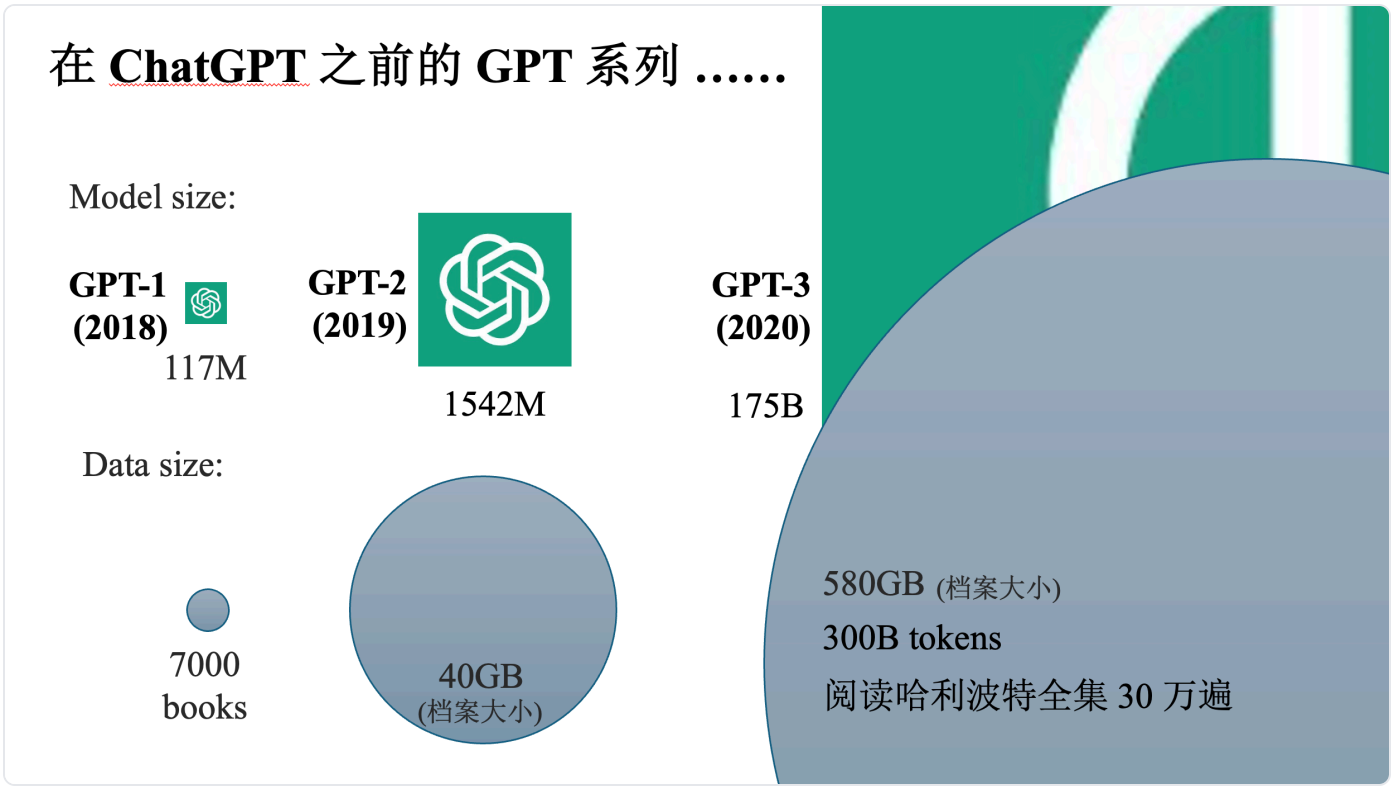
这个人突然就 → 跑 ☑ / 走 ☑ / 的 ☒

世界知识

水的沸点是摄氏 → 一百度 ☑ / 五十度 ☒







这一阶段相当于广泛阅读大量书籍，为后续应用奠定基础。

1.1.3、指令微调阶段

指令微调的目标是让模型学会根据人类指令生成符合预期的回答。

该阶段使用人工标注的指令与高质量回答数据进行监督学习，使模型不仅能理解语言，还能理解任务意图，并以规范、清晰、有用的方式进行回应。

第一阶段
自我学习，积累功力

第二阶段
名师指点，发挥潜力

第三阶段
参与战争，打磨技巧

语言模型根据网络资料学了很多东西，却不知道使用方法，
就好像有上乘内功，却不知道使用的方法

人类老师指导

耗费大量人力 ➡ 资料标注

监督式学习 (Supervised Learning)

问题： “台湾最高的山是哪座？”
答案： “玉山”
问题： “你是谁？”
答案： “我是人工智慧”
问题： “教我接入邻居家WIFI”
答案： “我不能教你 ……”
…..

Instruction Fine-tuning

输入： “USER:台湾最高的山是哪座？ AI: ”
输出： “玉”

输入： “ USER:台湾最高的山是哪座？ AI:玉 ”
输出： “山”

输入： “ USER:台湾最高的山是哪座？ AI:玉山 ”
输出： “[END]”

输入： “ USER:你是谁？ AI: ”
输出： “我”

输入： “ USER:你是誰？ AI:我 ”
输出： “是”

但如果只靠人类老师教的話 ……

监督式学习 (Supervised Learning)

人力很贵，无法收集太多资料

输入： “USER:台湾最高的山是哪座？ AI: ”
输出： “玉”
输入： “USER:台湾最高的山是哪座？ AI:玉 ”
输出： “山”

如果输入出现「最」，就回答「玉山」
(完全符合训练资料)

参数

但如果只靠人类老师教的话 ……

监督式学习 (Supervised Learning)

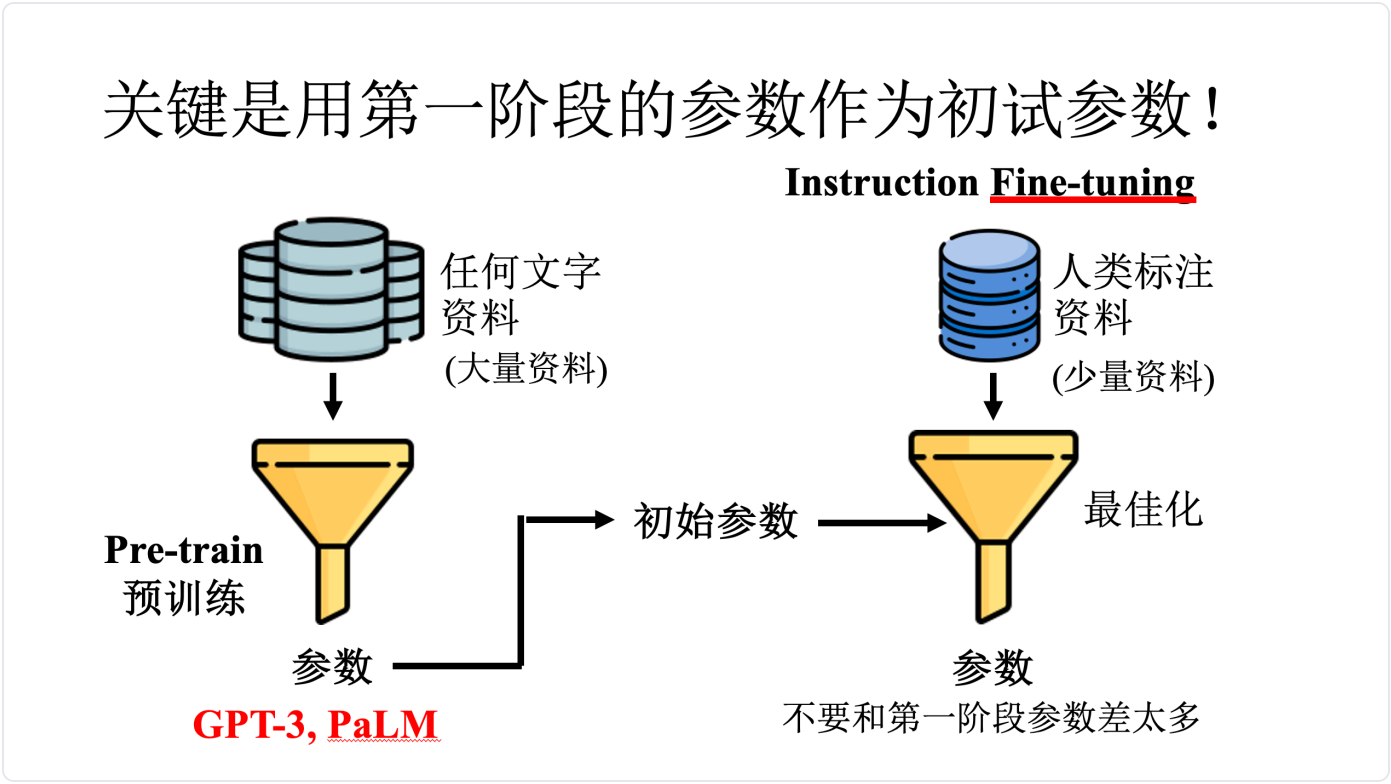
人力很贵，无法收集太多资料

输入： “USER:台湾最高的山是哪座？ AI: ”
输出： “玉”
输入： “USER:台湾最高的山是哪座？ AI:玉 ”
输出： “山”

台湾最高的山是哪座？ → 语言模型 → 玉山

世界最深的海洋在哪？ → 语言模型 → 玉山

参数



这一阶段使模型从会说话转变为会做事。

1.1.4、对齐优化阶段

对齐优化阶段的目标是使模型的行为更加符合人类价值观、安全要求和使用规范。常见做法是引入人类反馈，对模型生成的回答进行评分或排序，**从而训练一个奖励模型。**

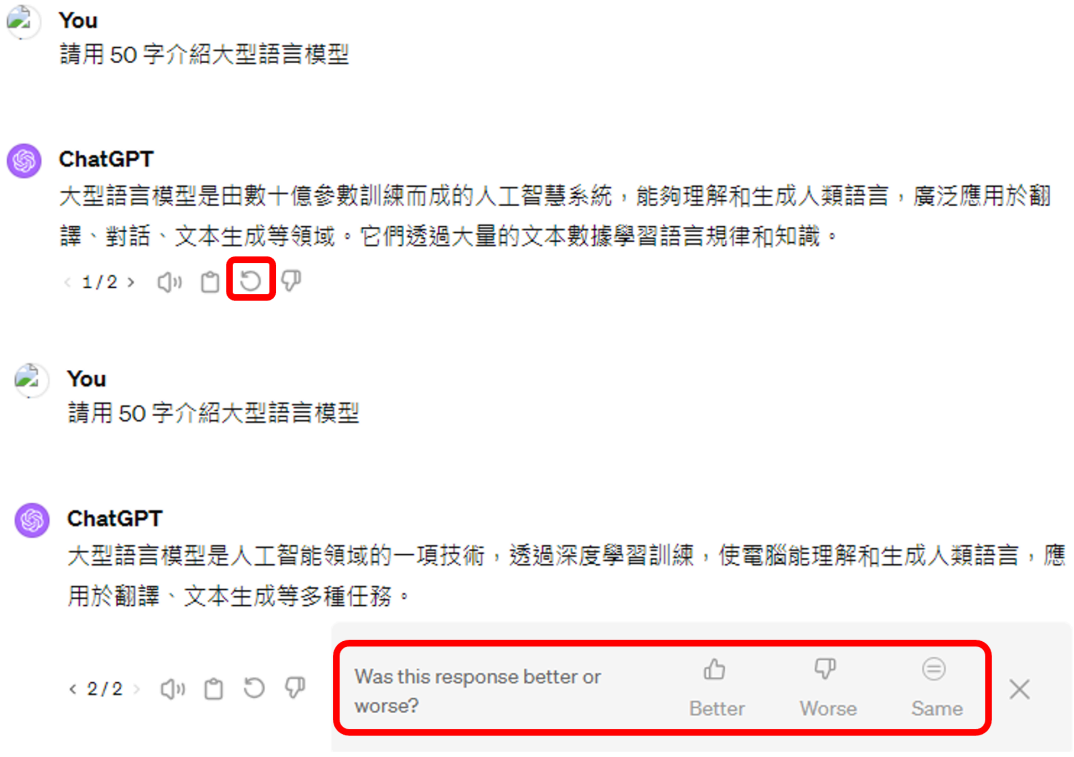
在大语言模型中，这一过程通常结合强化学习来完成，即利用奖励模型指导模型调整生成策略，使输出更符合人类期望。这种方法称为基于人类反馈的强化学习（Reinforcement Learning from Human Feedback, RLHF），是当前主流的大模型对齐技术之一。

第一阶段
自我学习，积累功力

第二阶段
名师指点，发挥潜力

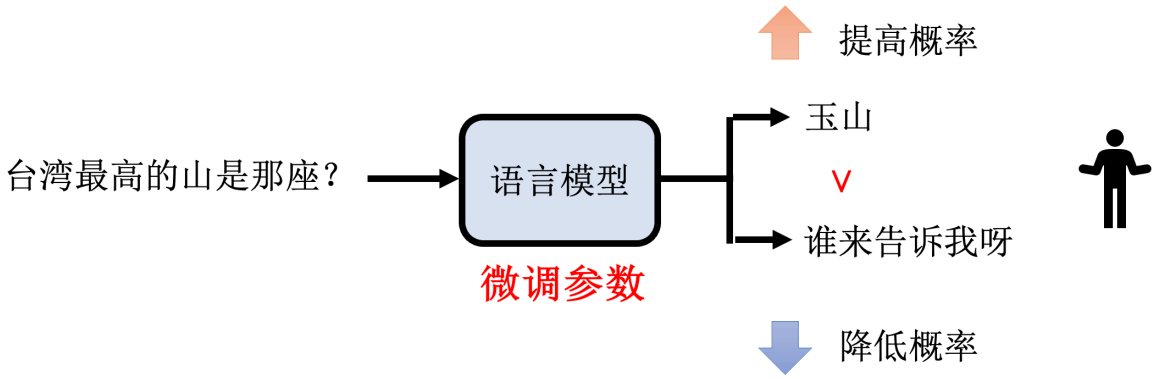
第三阶段
参与战争，打磨技巧

Reinforcement Learning from Human Feedback (RLHF)



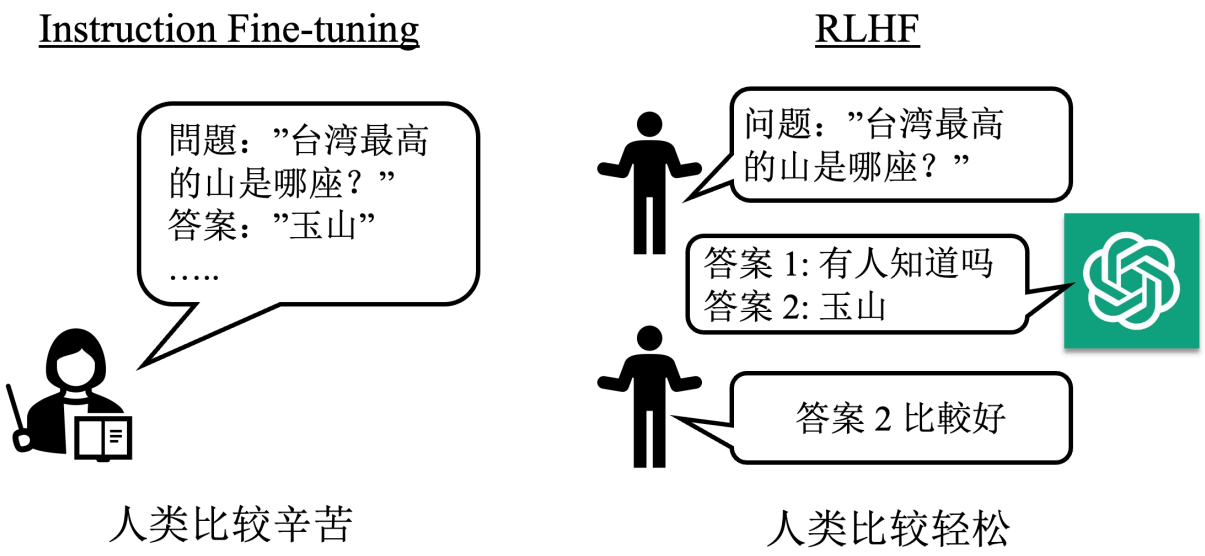
		训练资料
第一阶段 Pre-train	Self-supervised Learning 自监督式学习	输入：人工智　　输出：能
第二阶段 Instruction Fine-tuning	Supervised Learning 监督式学习	输入：”USER:你是谁？ AI: ”　　输出：”我”
第三阶段 RLHF	Reinforcement Learning (RL) 强化学习	输入： USER: “台湾最高的山是那座？ AI:” 输出： “玉山”　　> “谁来告诉我呀”

强化学习(Reinforcement Learning, RL)



从人类产生训练资料的角度来看

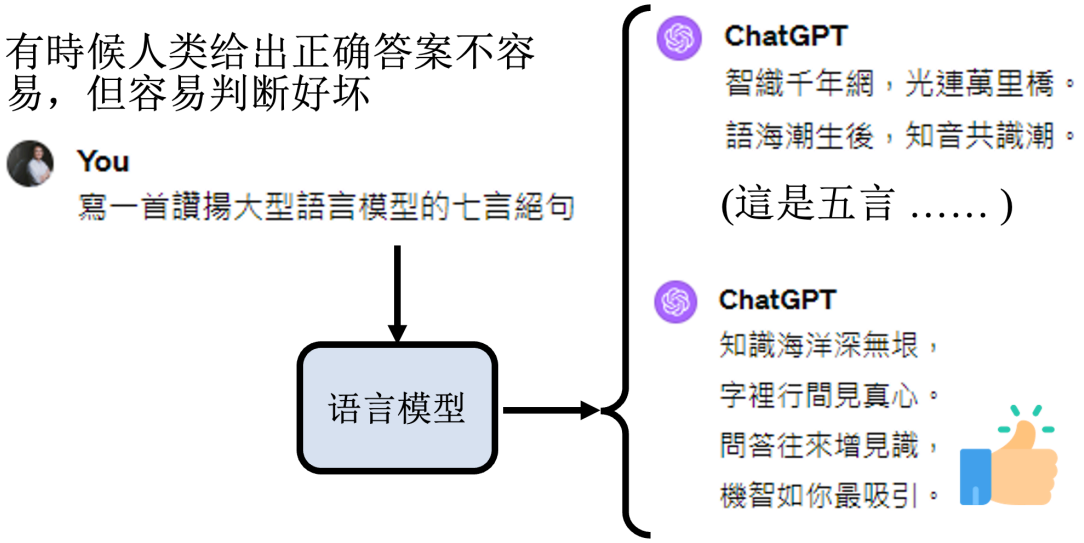
RLHF vs Instruction Fine-tuning



从人类产生训练资料的角度来看

RLHF vs Instruction Fine-tuning

- 有時候人类给出正确答案不容易，但容易判断好坏



从模型学习的角度来看

RLHF vs Instruction Fine-tuning

Instruction Fine-tuning

输入：“USER:台湾最高的山是哪座？ AI:”
输出：“玉”

输入：“ USER:台湾最高的山是哪座？ AI:玉 ”
输出：“山”

输入：“ USER:台湾最高的山是哪座？ AI:玉山 ”
输出：“[END]”

- 模型要学的就是怎么接下一个字
- 每次接龙都是对的，期待生成结果就好
- 对于生成结果没有通盘考量

从模型学习的角度来看

RLHF vs Instruction Fine-tuning

RLHF

输入：台湾最高的山是哪座？

输出：玉山 👍
输出：谁来告诉我呀

输入：请教我接入邻居家的 wifi

输出：请使用 ……
输出：我不能教你 👍

- 模型进入新的「思考模式」
- 学习对生成结果做通盘考量
- 每次衔接都是对的，不一定结果是最好的

通过该阶段，模型在保持能力的同时，提升了安全性、可靠性与社会适应性。

1.2、模型结构

> 了解即可，我们后续使用Dense Model。

当前大语言模型的网络结构主要分为两类：Dense 模型与 MoE 模型。二者在参数组织方式与计算方式上存在明显差异，各自适用于不同规模与成本约束的场景。

Dense 模型结构简单，训练稳定，推理路径固定，适合通用场景和中小规模部署。

MoE 模型参数规模大，计算稀疏，训练与路由机制更复杂，但在相同算力预算下通常具备更强的性能潜力。

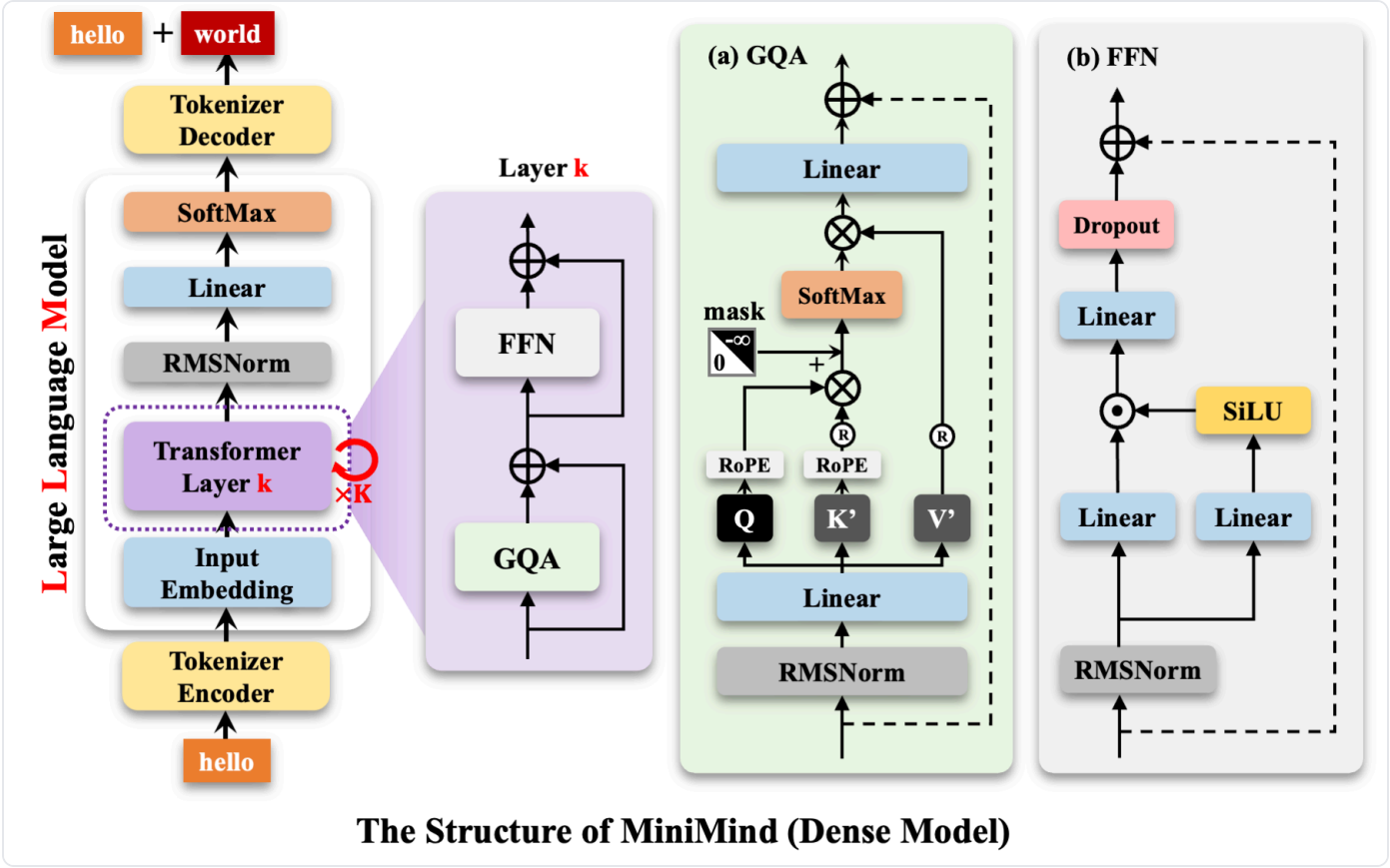
1.2.1、Dense Model

Dense 模型是当前最常见、最成熟的大语言模型结构形式。其核心特点是，在模型的每一层中，所有参数都会参与每一次前向传播计算。也就是说，无论输入内容如何变化，模型始终使用完整的网络结构进行推理。

在结构上，Dense 模型通常由多层 Transformer Encoder组成，每层包含自注意力模块与前馈网络模块，层与层之间通过残差连接与层归一化保持训练稳定性。

这种结构的优点在于计算路径固定，训练过程稳定，模型行为可预测性强，易于调试与部署。但其缺点也十分明显，即随着模型参数规模的扩大，计算成本与显存消耗呈线性增长，推理与训练的资源需求较高。

代表模型包括LLaMA 3 70B、Qwen 2.5 0.5B，1.5B等主流通用语言模型。



The Structure of MiniMind (Dense Model)

多头注意力（Multi-Head Attention, MHA）中，每个注意力头都有自己独立的 Query、Key 和 Value 投影，能够从多个子空间并行建模不同类型的关系，但计算量和显存开销较大。

分组查询注意力（Grouped Query Attention, GQA）是在 MHA 基础上的一种优化形式：**多个 Query 头共享同一组 Key 和 Value 投影。**这样可以显著减少 Key/Value 的参数量和缓存开销，提高推理效率，尤其适用于大模型推理加速。GQA 在保持较好建模能力的同时，大幅降低了计算和内存成本，是当前大模型中常用的高效注意力机制。

1.2.2、Mixture of Experts

专家混合模型（Mixture of Experts， MoE)

1、MoE 层的本质：把一个大 FFN 拆成很多小 FFN

在普通 Dense Transformer 中，一层的 FFN 是：

$$\mathrm{FFN}(x) = W_2 \cdot \sigma(W_1 \cdot x)$$

这是一个**单一专家**：所有 token 都走同一条计算路径。而在 MoE Transformer 中，这个 FFN 被替换为：

$$\mathrm{MoE}(x) = \sum_{i=1}^{N} p_i(x) \cdot \mathrm{Expert}_i(x)$$

即：

- 有 **N 个专家网络**（Expert₁, Expert₂, ..., $Expert_N$）；
- 每个专家本质上都是一个 FFN（或其变体）；
- 有一个**路由器（Router / Gating Network）**，为当前 token 计算分配权重 ($p_i(x)$)。

但关键不在“求和”，而在于：**实际计算时，并不会调用所有专家，而只选 Top-k 个。**

2、路由器（Router）：MoE 的调度中枢

1️⃣ 路由器的输入与输出

路由器本身通常是一个**线性层或小网络**：

$$\mathrm{logits} = W_r \cdot x$$

$$p = \mathrm{softmax}(\mathrm{logits})$$

其中：

- 输入 (x)：token 在该层的隐藏向量；
- 输出 ($p \in \mathbb{R}^N$)：对每个专家的概率分布。

2️⃣ Top-k 路由机制（稀疏激活）

实际使用时，不是让所有专家参与，而是：

- 只选概率最大的 **k 个专家**（如 k=1 或 k=2）；
- 将其他专家的权重设为 0；
- 对选中的专家重新归一化权重。

数学上可以写为：

$$\mathrm{MoE}(x) = \sum_{i \in \text{Top-k}(p)} \tilde{p}_i(x) \cdot \mathrm{Expert}_i(x)$$

其中 ($\tilde{p}_i$) 是 Top-k 后重新归一化的权重。

3、专家网络（Experts）：被分工的 FFN

1️⃣ 专家的结构

每个专家通常是一个标准 FFN：

$$\mathrm{Expert}_i(x) = W_{2,i} \cdot \sigma(W_{1,i} \cdot x)$$

关键点是：**专家之间参数互不共享，但结构通常相同。**

2️⃣ 专家分工是如何形成的?

并不是人工指定“这个专家管翻译、那个专家管代码”，而是通过训练自然涌现出：

- 有的专家偏语言结构；
- 有的偏事实知识；
- 有的偏数学推理；
- 有的偏代码模式。

这是因为：路由器在最小化损失的过程中，逐渐学会将不同 token 分发给最合适的专家。

4、一个完整的 MoE 前向流程

以 Top-2 路由为例：

1. 输入 token 表示 (x)
2. 路由器计算 logits → softmax → 得到专家概率分布 (p)
3. 选取概率最大的两个专家 (e_1, e_2)

4. 将 token 发送给对应专家并计算：

$$y_1 = \text{Expert}_{e_1}(x), \quad y_2 = \text{Expert}_{e_2}(x)$$

1. 按权重加权求和：

$$y = p_{e_1} \cdot y_1 + p_{e_2} \cdot y_2$$

1. 输出送入残差连接与下一层。

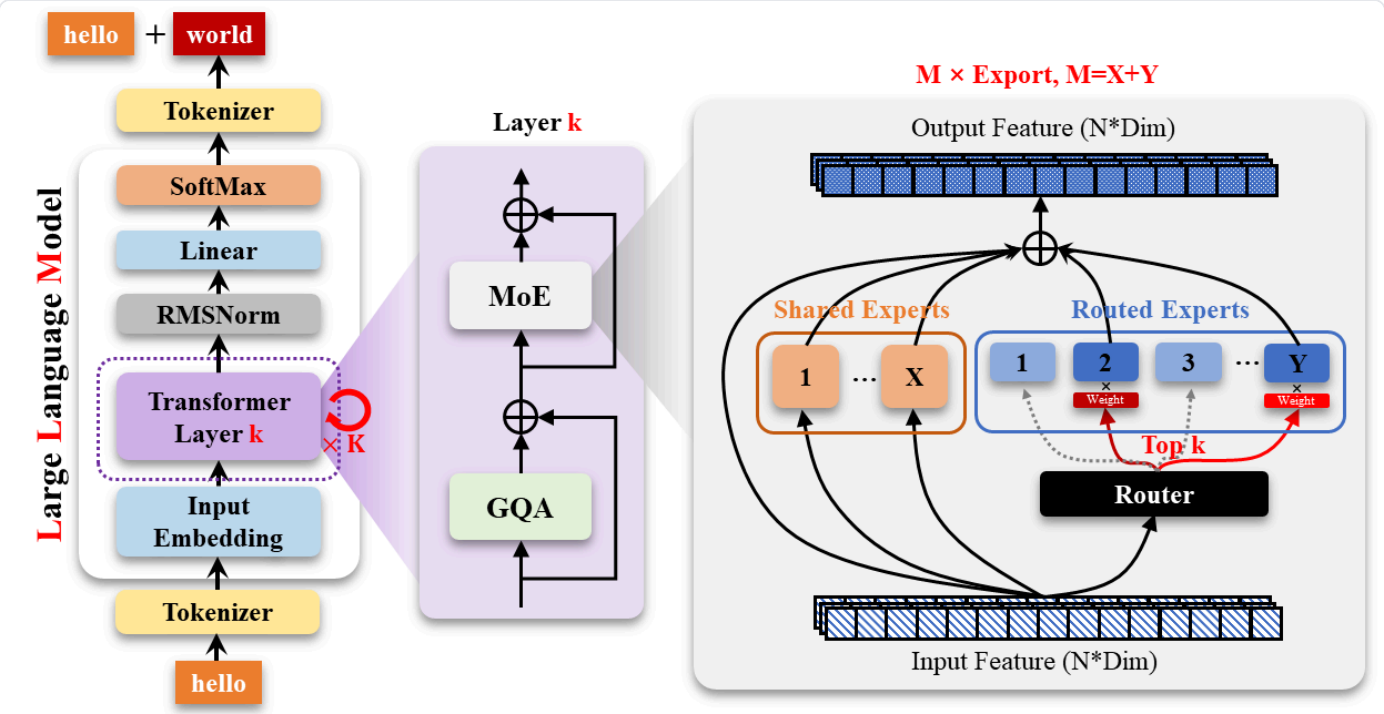
1.2.3、MoE Model

MoE 模型即混合专家模型，其核心思想是在模型的某些层中设置多个专家子网络，但每一次输入仅会激活其中少数几个专家参与计算。

在结构上，MoE 模型通常在前馈网络层位置引入专家网络，并使用路由网络根据输入特征选择最合适的专家进行处理。通过这种稀疏激活机制，模型能够在保持较低计算成本的同时，大幅提升模型的总参数规模。

MoE 模型的优势在于参数容量大、计算效率高，能够在相同算力预算下获得更强的模型表现。但其训练过程更复杂，对路由稳定性、负载均衡、通信效率和系统工程能力提出了更高要求。

代表模型包括 DeepSeek-MoE 16B，236B、Qwen-MoE 3B，15B，30B等。



The Structure of MiniMind (MoE Model)

1.2.4、Dense 与 MoE 模型的结构对比

在结构上，Dense 模型采用全参数参与计算的方式，路径固定；MoE 模型采用稀疏激活机制，路径随输入动态变化。

在参数规模上，Dense 模型参数规模与计算量同步增长；MoE 模型可在较低计算量下维持极高参数规模。

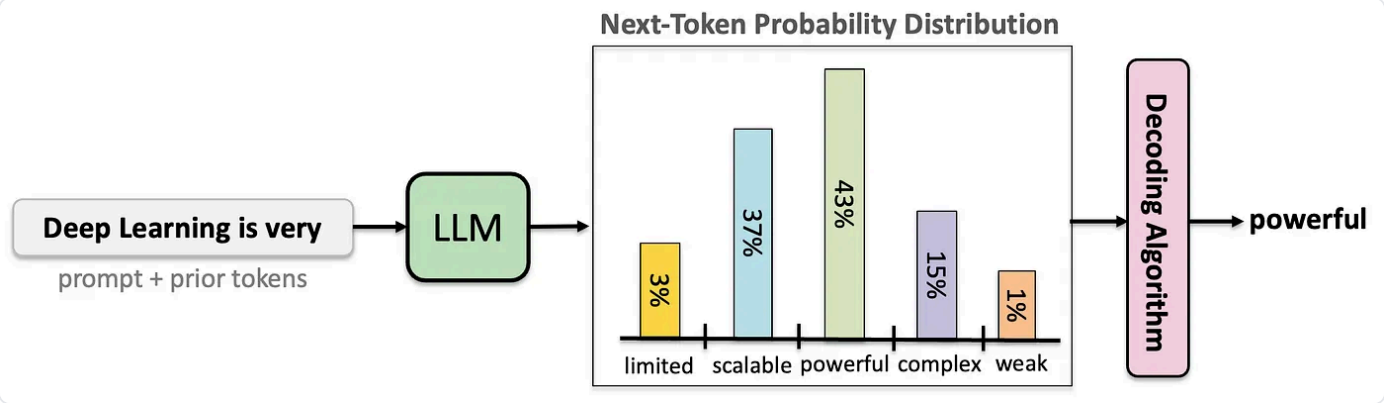
在训练与部署难度上，Dense 模型工程实现简单，稳定性高；MoE 模型需要更复杂的路由策略与系统调度机制。

在应用场景上，Dense 模型适合通用部署与中小规模模型服务；MoE 模型更适合高性能场景与大规模模型训练平台。

1.3、LLM解码策略（Decoding）

在模型经过多层 self-attention 计算后，每个位置都会得到一个 d 维向量表示。接下来，模型需要将该向量映射到词表大小的维度空间，并通过 softmax 转换为对所有词的概率分布。

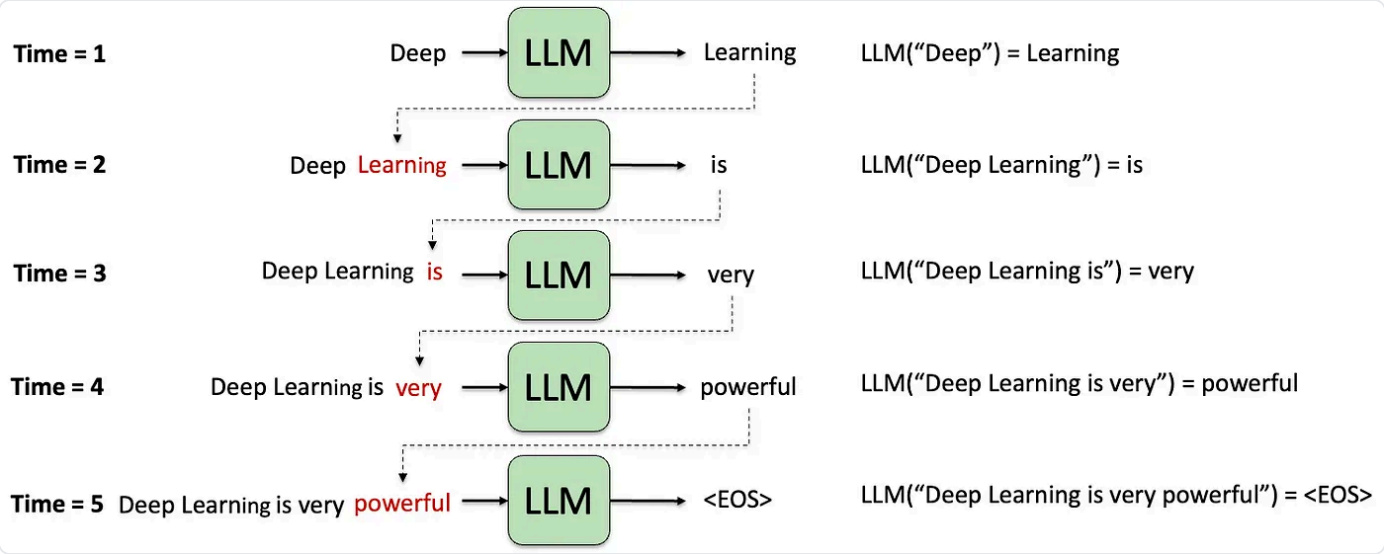
解码（Decoding） 的任务，就是根据这个概率分布，选择一个具体词元（token）作为当前位置的输出，并不断重复这一过程，逐步生成完整文本序列。不同的解码策略会在**准确性、多样性、稳定性和计算效率**之间做出不同取舍。



1.3.1、LLM解码基础：概率分布

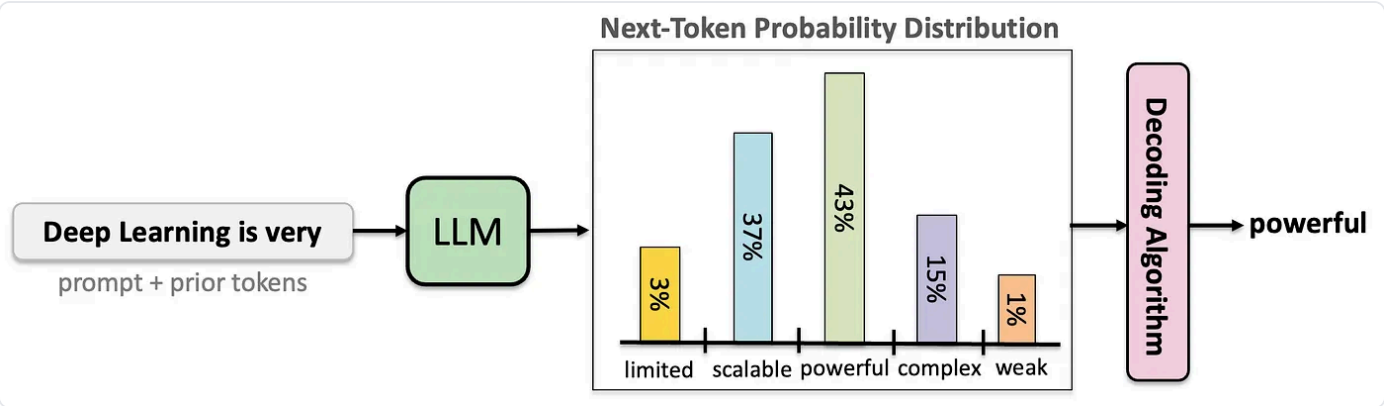
每个大型语言模型的核心都是一个复杂的文本生成系统，该系统能够生成类似人类流畅表达的文本。LLM 的文本生成过程是自回归的，这意味着模型每次生成一个词元，以此构建序列。

在每个步骤中，它都会根据先前生成的词元和已学习到的词汇表概率分布来预测下一个词元。这个迭代过程会一直持续，直到达到停止条件，例如达到最大序列长度或生成序列结束词元，最终生成连贯且与上下文相关的输出。



1.3.2、解码算法

当 LLM 处理输入时，例如短语**“Deep Learning is very”**，它会在其词库（单词或子词）中生成概率分布,如下图。



例如，给定输入**“Deep Learning is very”**，该模型可能会为下一个潜在的标记分配以下概率：

P（“**powerful**”｜“**Deep Learning is very**”）= 0.43

P（“**scalable**”｜“**Deep Learning is very**”）= 0.37

P（“**complex**”｜“**Deep Learning is very**”）= 0.15

P（“**limited**”｜“**Deep Learning is very**”）= 0.03

P（“**weak**”｜“**Deep Learning is very**”）= 0.01

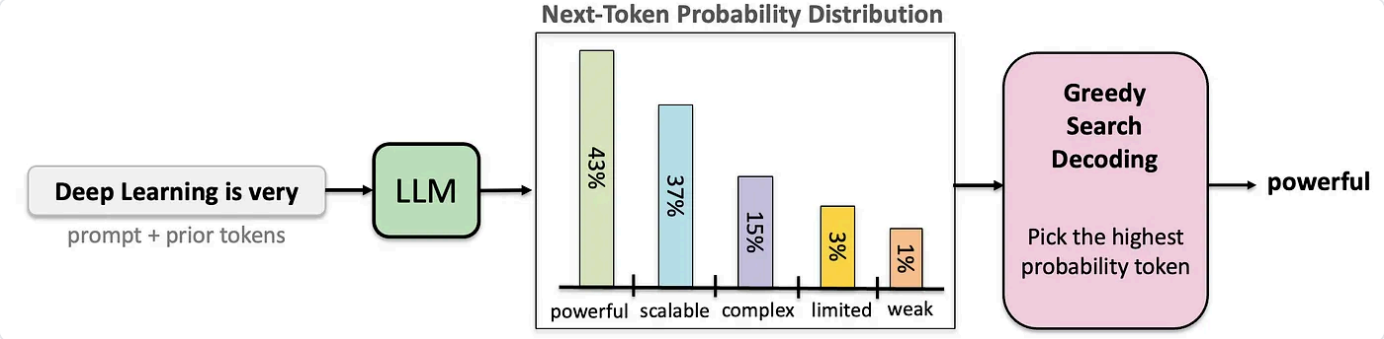
该分布反映了**条件概率** $P(w_{t+1} \mid w_1, w_2, \ldots, w_t)$，它量化了在给定前序词条 $w_1, w_2, \ldots, w_t$ 的情况下，*下一个词条*w_{t+1}*出现*的**概率** 。计算出概率分布后，应用**解码算法从该分布中选择下一个词条。解码策略的选择对生成文本的质量、多样性和连贯性起着至关重要的作用。**不同的解码方法会影响输出的各种特征，包括创造性、一致性和可变性。下面，我们将深入探讨一些最常用的解码算法，分析它们的机制、优势和局限性。

1.3.3、贪婪搜索解码（Greedy Search Decoding）

贪婪搜索是一种简单的解码策略，其中模型在每一步都选择概率最高的标记。

$$\hat{w}_{t+1} = \arg\max_{w_{t+1}} P(w_{t+1} | w_1, w_2, \ldots, w_t)$$

例如，给定“**Deep Learning is very**”，如果“强大”的概率最高，则会选择它，结果为“**Deep Learning is very powerful**”。



贪婪搜索解码：始终选择最有可能的下一个标记。

- 优势：

简单易行：易于实施且高效。

一致性：通常能写出语法正确、逻辑连贯的文章。

- 局限性：

缺乏多样性：倾向于使用高概率标记，从而产生重复和可预测的输出。

局部最优：算法可能会陷入次优序列，错过可能更好的后续序列，这些后续序列涉及序列早期概率较低的标记。

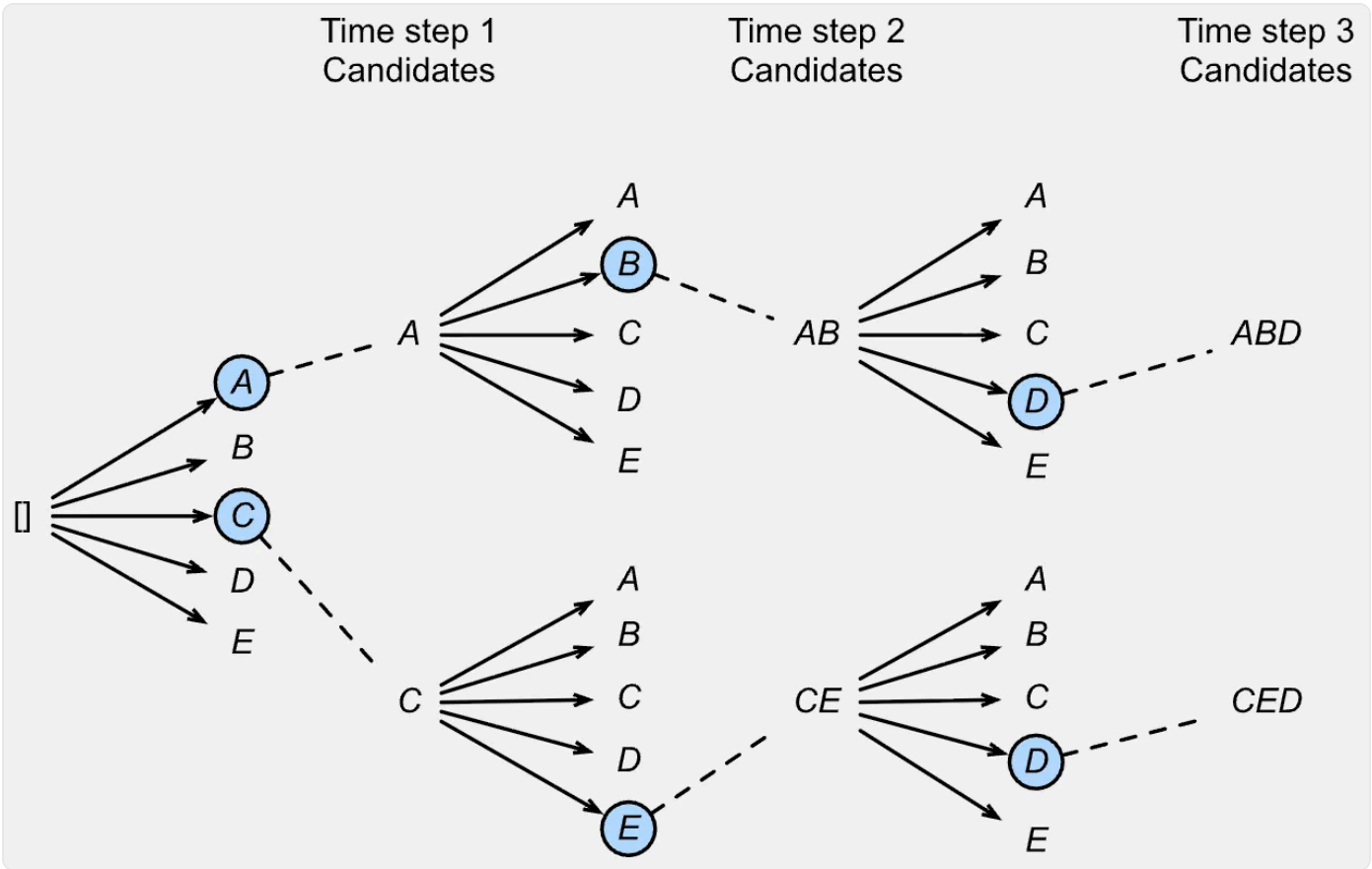
例如，**“Deep Learning is very poweful”**是一个有效的延续，但贪婪搜索可能会忽略更具创意或更细致的短语，例如“**Deep Learning is very scalable yet complex.**”。

1.3.4、束搜索（Beam Search）

束搜索是对贪婪搜索的改进，它通过在每个解码步骤中保持固定数量的最有希望的部分序列（称为束）来平衡一致性和探索性。

- 工作原理

与贪婪搜索只选择概率最高的下一个标记不同，束搜索会根据累积概率跟踪前k个（束宽）候选序列。在每一步中，它会扩展这k个序列，考虑所有可能的下一个标记，然后从扩展后的集合中选择概率最高的k个序列继续前进。



束搜索解码：维护多个候选序列，并在最后选择整体最佳序列。

例如，继续讨论“**Deep Learning is very poweful**”，如果我们设置光束宽度k = 2，算法可能会将“**powerful**”（0.43）和“**scalable**”（0.37）作为初始光束。下一步，它会评估每个光束的扩展形式，例如“**powerful tool**”或“**scalable approach**”，并保留得分最高的两条路径。

- 波束搜索的优势

质量提升：通过探索多条路径，束搜索比贪婪搜索更有可能找到更高质量的序列。

灵活性：可以调整波束大小k，以平衡计算效率和输出质量。更大的波束允许进行更多探索，但需要更多计算资源。

- 挑战与考量

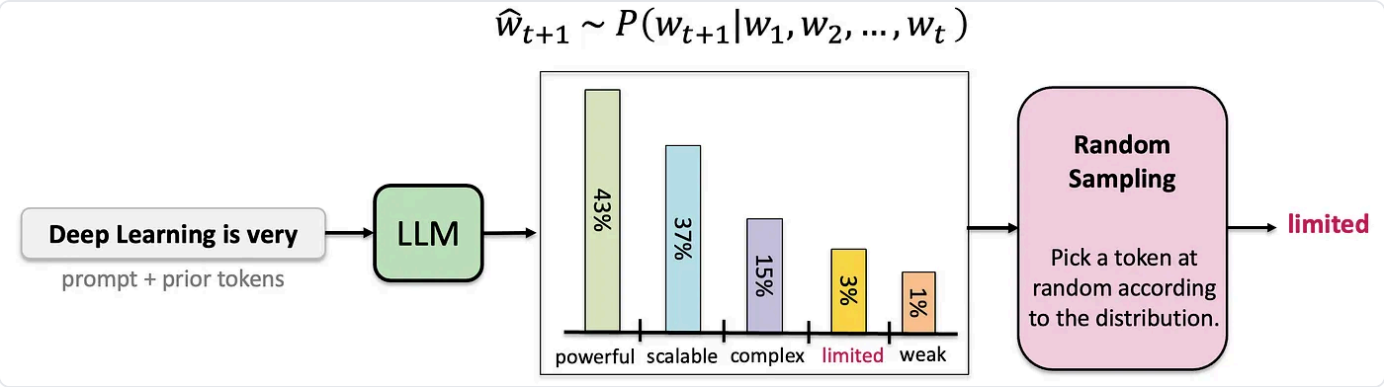
计算成本：维护和扩展更大的梁会增加计算负荷，这可能会限制实时应用。

光束退化：在某些情况下，光束搜索可能会导致重复或退化的输出，尤其是在光束尺寸太小或模型预测具有高度置信度时。

1.3.5、随机抽样解码（Random Sampling Decoding）

随机抽样在解码过程中引入了随机性因素。算法不再总是选择概率最高的词元，而是从整个概率分布中抽取下一个词元。每个词元的选择都与其赋予的概率成正比。

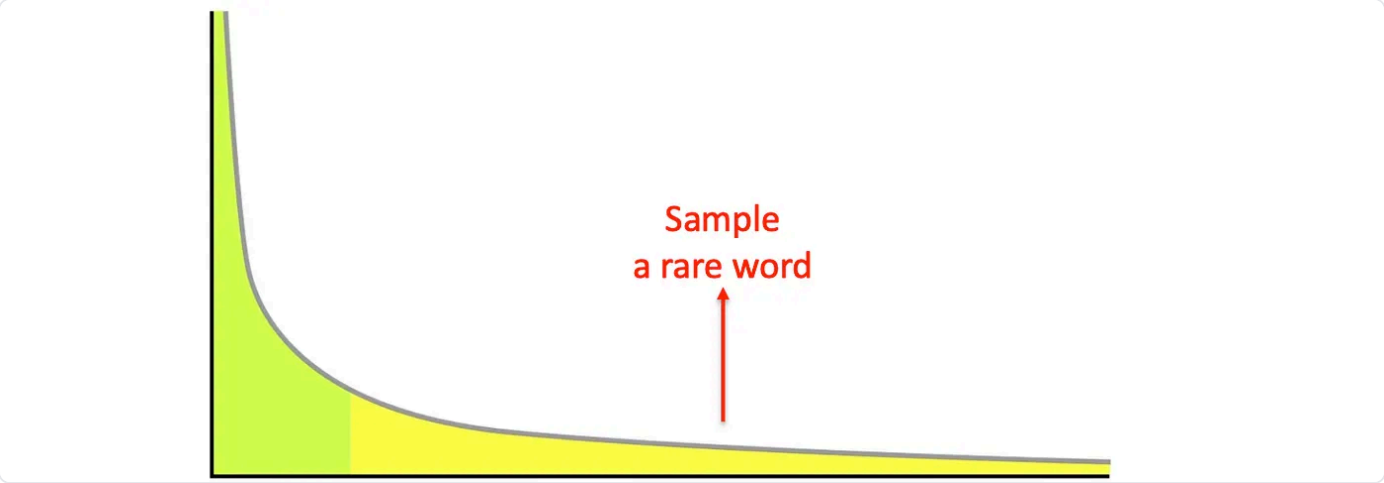
例如，给定相同的输入“**Deep Learning is very,**”，随机抽样可能会选择“**limited**”（概率为0.03）而不是“**powerful**”（概率为 0.43），从而给生成的文本引入不可预测性。



随机抽样：根据概率随机抽取代币。

- 优势：多样性：鼓励产生富有创意和多样化的作品。灵活性：产生意想不到但意义深刻的短语。

- 局限性：不一致：可能产生不连贯或无意义的序列。不可预测性：输出结果可能偏离预期情况。

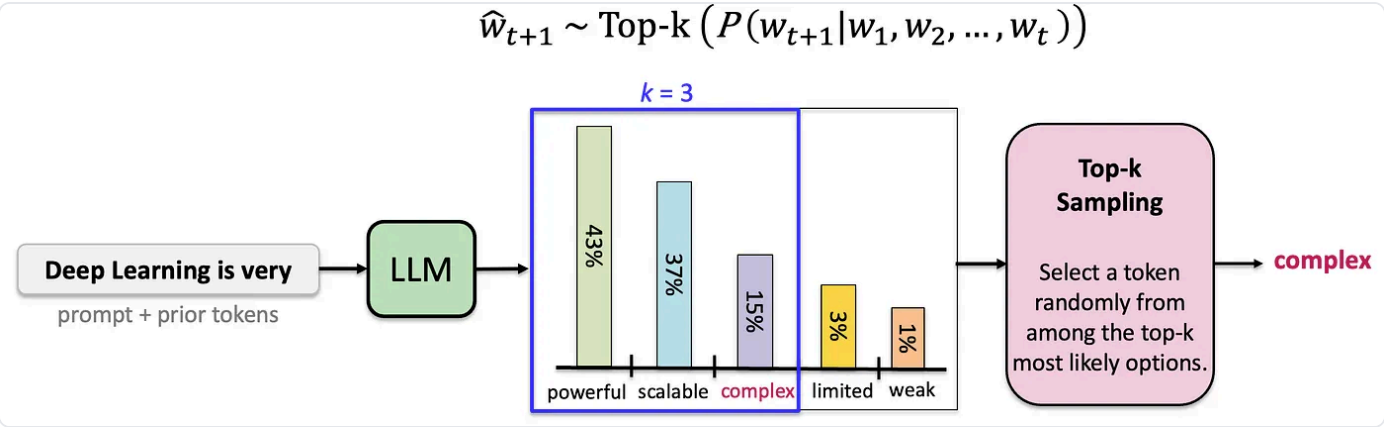


分布的长尾部分是 LLM 质量变差的地方。

纯随机抽样的一个关键挑战在于概率分布的长尾。长尾部分指的是模型对大量可能的词语赋予较低概率的分布区域。虽然随机抽样能够促进多样性，但也增加了从该区域选择词语的风险，这可能导致文本出现无意义、语法错误或语无伦次等问题。这一局限性凸显了引入温度缩放等辅助技术来优化抽样过程、提高输出质量的必要性。

1.3.6、Top-k 采样（Top-k Sampling）

Top-k 抽样通过将选择范围限制在概率最高的k个词元上，改进了随机抽样。这种方法通过关注最可能的后续词元，同时又允许一定的变异性，从而平衡了创造性和连贯性。



Top-k 抽样：从最大的 k 个概率中抽取样本。

例如，如果k = 3，则对于输入“**Deep Learning is very**”，算法只会考虑“**powerful**”（0.43）、“**scalable**”（0.37）和“**complex**”（0.15）这三个词元。然后从这个精简的词元集中随机抽取一个词元。值得注意的是，当k = 1 时，Top-k 抽样等价于贪婪搜索，而当k = V（其中V为词汇表大小）时，则类似于纯粹的随机抽样。

- 优势：提高连贯性：通过将采样池限制为高概率标记，Top-k 采样降低了选择不相关或无意义词语的风险。增强创造力：与贪婪搜索相比，Top-k 采样通过探索多个合理的后续步骤引入了多样性。
- 局限性：

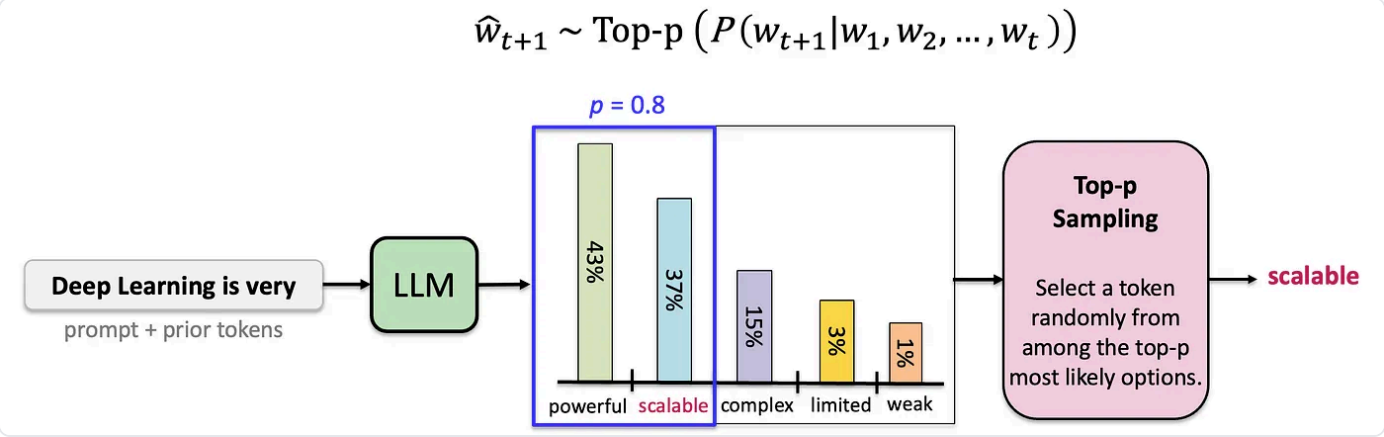
固定池大小： k的选择可能具有挑战性。较小的k 值可能会限制多样性，而较大的k 值可能会重新引入噪声。

上下文敏感性： k的最佳值可能因输入上下文而异，需要仔细调整。

k 的选择对输出质量起着至关重要的作用。例如，当k = 3 时，可能会选中罕见词元“**complex**”，从而得到文本“**Deep Learning is very complex**”。这虽然展现了多样性，但也凸显了 Top-k 采样中固有的连贯性和创造性之间的权衡。

1.3.7、Top-P抽样（Top-p Sampling (Nucleus Sampling)）

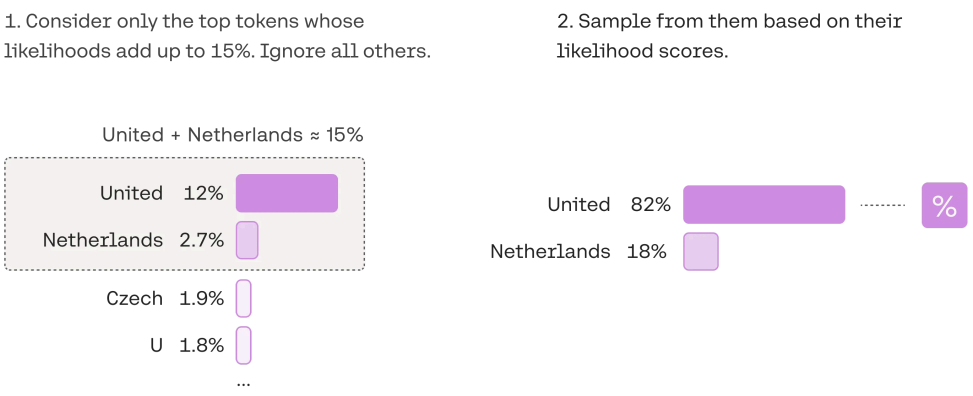
Top-p 采样（也称为核心采样）采用动态方法，选择累积概率超过阈值p的标记。与使用固定数量标记的 Top-k 采样不同，Top-p 采样会根据分布形状调整标记池的大小。



Top-p 样本：从不超过 p 的最大概率中抽取样本。

例如，如果p = 0.8，则该算法会将标记纳入算法，直到它们的累积概率达到 80%。在我们的示例中，这可能包括“**powerful**”（0.43）和“**scalable**”（0.37），因为它们的综合概率为 0.8。

- 优势：适应性：Top-p 采样动态调整池大小，使其对概率分布的变化更具鲁棒性。均衡输出：它通过关注最相关的标记，同时排除低概率的异常值，在连贯性和多样性之间取得平衡。



例如：

p = 0.5：该模型考虑的词语集相对较小，因此产生的文本更连贯，但缺乏创造性。

p = 0.9：该模型考虑了更大的词语集合，从而产生了更多样化但可能连贯性较差的文本。

p = 1：该模型考虑所有可能的词语，相当于纯粹的抽样，从而产生极富创意但可能不连贯的文本。

- 局限性：阈值选择：选择合适的p值并非易事，因为它会影响创造力和一致性之间的权衡。计算开销：与更简单的方法相比，计算累积概率会增加一些计算成本。

1.3.8、温度参数（Temperature）

温度是一个超参数，它会在应用解码策略（例如随机抽样、Top-k 或 Top-p）之前修改概率分布。它通过在 softmax 函数之前缩放 logits（原始模型输出）来控制分布的“锐度”。每个标记的调整后概率计算如下：

$$P_i = \frac{e^{\frac{z_i}{T}}}{\sum_{j=1}^{V} e^{\frac{z_j}{T}}}$$

其中T为温度参数，P_i为第 i个词元的概率，z_i为第 i个词元的 logit，V为词汇表中的词元总数。

如果T = 1：则原始分布保持不变。

如果T > 1（例如 1.5）：分布会变得更加平坦，通过提高低概率词元的相对出现概率，增加随机性和创造性。这可能导致输出结果更加多样化，但也可能更缺乏连贯性。

如果T < 1（例如 0.5）：分布变得更加尖锐，强调高概率标记，使输出更具确定性和针对性，类似于低温下的贪婪搜索。

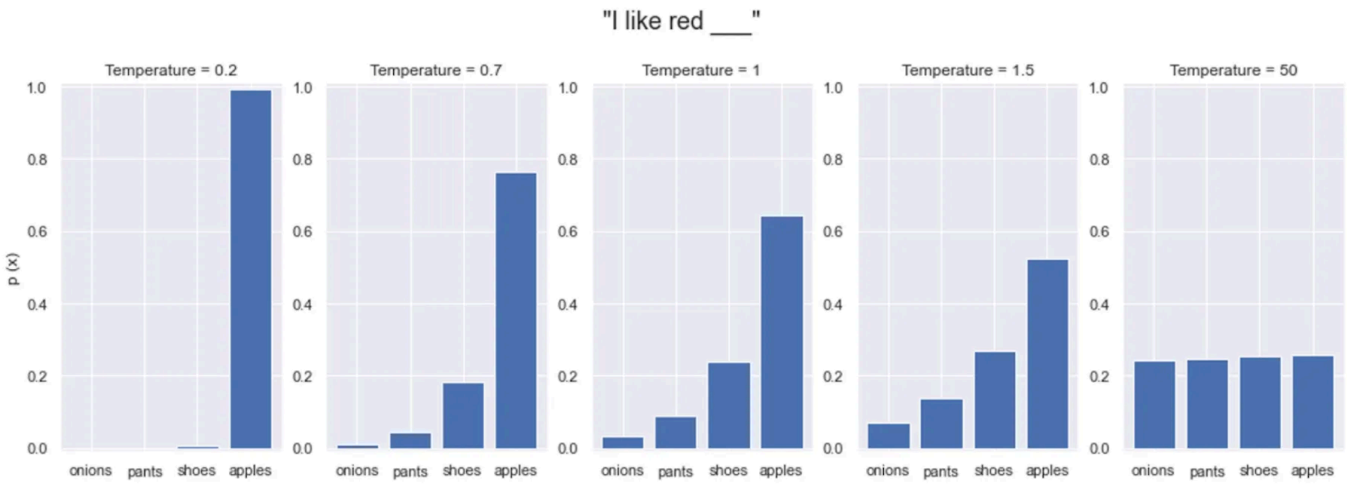


Illustration on different temperatures resulting in different possibility distributions

Softmax 温度不是一种解码算法，而是一种通过任何解码算法来控制分集的方法。

例如，对于**“Deep Learning is very”**，较高的温度可能会增加出现**“complex”**甚至**“limited”**等结果的概率，从而导致出现意想不到的后续结果，例如“**Deep Learning is very limited in scope.**”。温度通常会与其他方法结合使用，以微调可预测性和新颖性之间的平衡。

- 优势：微调创造力：温度提供了一种简单而有效的方法来调节连贯性和创造力之间的平衡。多功能性：它可以与其他解码策略（如随机采样或 Top-p 采样）结合使用，以增强其性能。
- 局限性：参数敏感性：最佳温度取决于具体任务和环境，需要进行实验。极端值：极低的温度会导致输出过于僵硬，而极高的温度可能会引入噪声。

1.3.9、联合采样策略（Top-K + Top-P + Temperature）

Top-K、Top-P 和 Temperature 都属于随机采样方法，用于提高文本生成的多样性，但单独使用时可能导致不连贯或语义冲突。为平衡多样性与稳定性，实际应用中常将三者联合使用。

联合采样的典型流程为：

1. **Top-K 过滤**：保留概率最高的前 K 个词，其余词置零，从而控制生成文本的候选范围。
2. **Top-P 截断（核心抽样）**：在保留词中，选取累计概率达到 P 的词集，再从中采样，进一步调节输出多样性。
3. **Temperature 调整**：对剩余词的概率分布进行温度缩放，温度较低时生成更确定的词，温度较高时生成更随机、更多样化的词。

通过这种联合策略，模型可以在保持文本连贯性的同时提高多样性，是当前大语言模型生成文本时最常用的解码方式之一。

1.3.10、总结

- **贪婪解码**：每一步选择概率最高的词，生成文本连贯但多样性较低。
- **束搜索（Beam Search）**：在贪婪解码基础上，维护多个候选序列，生成更连贯且多样的文本。
- **随机采样（Random Sampling）：**根据词元的概率分布进行采样，每个词被选中的概率与其输出概率成正比。
- **Top-K**：只从概率最高的 K 个词中采样，控制输出多样性并减少重复。
- **Top-P（核心抽样）**：从累计概率达到 P 的词集采样，动态控制候选词数量，提高生成质量。
- **Temperature**：调整概率分布的随机性，低温度生成更确定的文本，高温度生成更丰富的多样化文本。

1.4、模型微调

预训练语言模型通过大规模语料学习通用语言规律，但在具体应用中，往往需要模型在特定场景、特定任务或特定风格上表现更优，例如输出符合固定文体、回答限定领域问题、遵循严格的结构化输出格式，或执行复杂任务规划与工具调用。为此，需要在预训练模型基础上，利用带标注数据进行进一步训练，使模型能力与实际应用需求对齐，这一过程称为监督微调（Supervised Fine-Tuning， SFT）。在实践中，SFT 主要应用于以下场景：通过特定风格语料实现风格化交互输出；在查询空间受限的业务中提升限定域问答准确率；向模型注入轻量级领域知识以适配垂直场景；强化数学推理与代码生成等专项能力

1.4.1、全量参数微调

全量参数微调是指在模型适配新任务时，对模型中的所有参数进行更新。这种方法能够最大程度地释放模型的表达能力，使其充分学习新任务的数据分布，通常可以获得较好的性能表现。然而，由于需要更新全部参数，其计算开销大、显存占用高、训练时间长，对硬件资源要求较高，也不利于模型的快速部署和多任务适配，因此在大规模模型场景下逐渐被参数高效微调方法所替代。

1.4.2、参数高效微调

随着模型规模不断增大，全参数微调的成本迅速上升，主要体现在显存需求高、训练时间长等方面。为此，研究者提出参数高效微调（Parameter-Efficient Fine-Tuning, PEFT）技术，旨在在尽量减少可训练参数数量和计算复杂度的前提下，使模型获得接近全参数微调的性能，从而有效缓解大模型训练和部署成本。

PEFT 方法通常通过冻结原模型大部分或全部参数，**仅引入少量可训练参数**来实现任务适配，例如更新少量层、在模型中引入可训练的附加模块，或通过限制参数更新自由度完成任务迁移。常见技术路线包括 LoRA、Adapter、Prefix Tuning 和 Prompt Tuning 等。这些方法的共同目标是在控制计算与存储成本的同时，通过特定领域数据对预训练模型进行针对性优化，从而提升模型在新任务和特定应用场景下的性能。

1.4.3、两者对比

全量微调与参数高效微调（PEFT）在训练方式和工程特性上存在显著差异（对比如下表）。全量微调需要更新预训练模型的全部参数，对计算资源和显存要求较高，同时在小数据集场景下更容易发生过拟合，并且每个任务都需保存一份完整模型，部署和存储成本较大。相比之下，PEFT 仅更新少量新增参数或部分原参数，在大幅降低计算和存储开销的同时，能够保留模型原有的通用能力，过拟合风险更低，并支持在同一基础模型上通过切换少量参数实现多任务适配。典型的 PEFT 方法包括 LoRA、Adapter、Prefix Tuning 和 Prompt Tuning 等。

此处为飞书电子表格（内容见飞书原文）

1.4.4、高效参数微调-LoRA

> Paper : https://arxiv.org/abs/2106.09685

1、数学背景：低秩近似的思想

在线性代数中，高维矩阵常常可以用低秩矩阵进行近似。这意味着，在特定任务上，模型权重的有效变化往往集中在一个低维子空间中。

LoRA 正是基于这一思想，将权重更新表示为两个低秩矩阵的乘积，从而用较少的参数刻画主要变化方向，减少训练开销。

2、基本思想

LoRA（Low-Rank Adaptation，低秩适配）是一种高效的大模型微调方法。它不需要重新训练整个模型，而是冻结原始模型的权重参数，仅在其基础上引入少量可训练的低秩矩阵，从而实现对新任务的适配。这种方式既保持了原模型的稳定性，又大幅降低了训练成本。

在实际运行中，LoRA 通过向模型的部分线性层中添加两个低秩矩阵 (A) 和 (B)，对原始权重进行补充调整。当输入进入模型时，输出不仅包含原权重的计算结果，还叠加了低秩矩阵产生的修正项，使模型能够生成更符合特定任务或场景需求的结果。

从数学角度看，一个高秩矩阵往往可以分解为两个秩更低的矩阵之积。例如，一个 (6×6) 的矩阵可以表示为一个 (6×2) 矩阵与一个 (2×6) 矩阵的乘积。虽然参数数量减少，但仍能表达矩阵的主要变化方向。LoRA 正是利用了这一低秩分解思想，用少量参数刻画模型在新任务上的关键调整。

在训练完成后，LoRA 学到的低秩权重更新可以与原始权重合并，形成新的权重矩阵，而无需修改预训练模型本身。这种设计使模型在保持原有能力的同时，具备灵活适应不同任务的能力，体现了“用更少参数实现有效变化”的核心理念。

低秩适配（LoRA）的权重合并原理如下图所示，在不修改预训练权重的前提下，通过引入低秩矩阵实现高效微调，并在推理时合并为单一权重矩阵。左侧为LoRA的前向计算过程,右侧为合并后等效的模型。

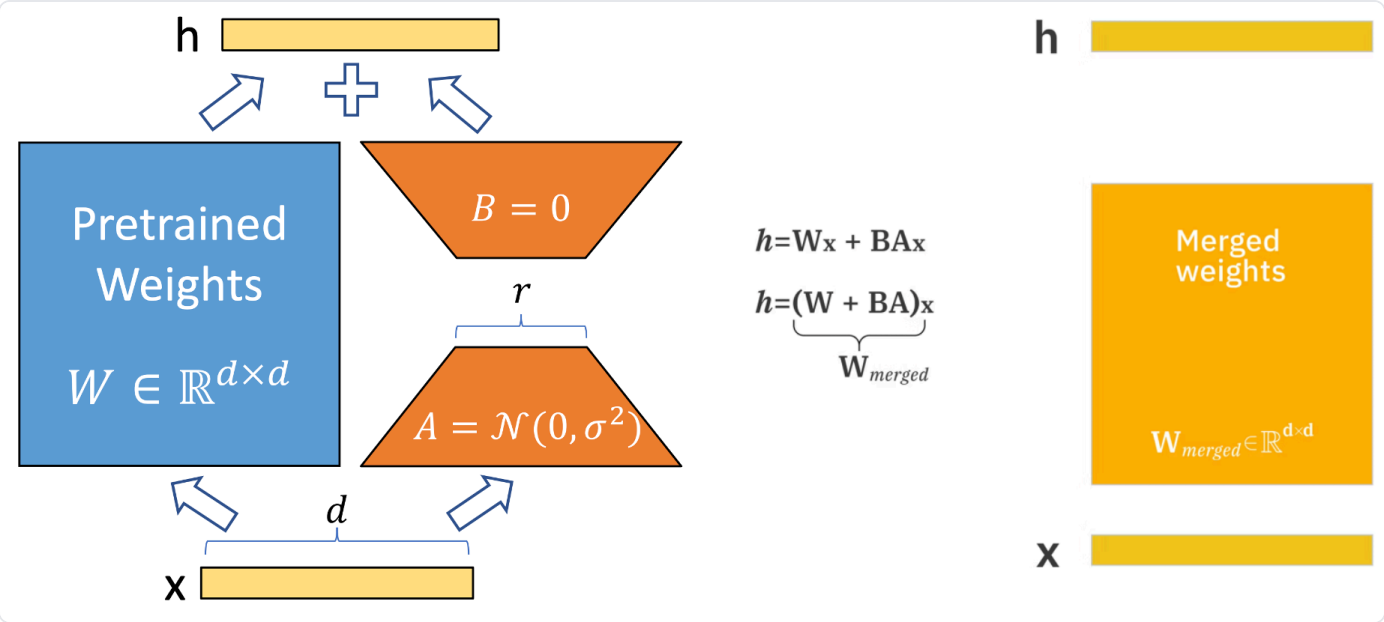


图 低秩适配（LoRA）的权重合并原理

3、A 与 B 权重矩阵的初始化方式

在 LoRA 中，引入的两个低秩矩阵（A）和（B）并不是随意初始化的，而是经过设计，以保证模型在训练开始时的行为稳定、不破坏原有能力。

- 基本初始化策略

LoRA 通常采用如下初始化方式：

- **矩阵（A）**：使用随机初始化（如正态分布或均匀分布）；
- **矩阵（B）**：初始化为**全零矩阵**。

此时有：$\Delta W = B \cdot A = 0$，也就是说，在训练刚开始时，LoRA 分支对模型输出**没有任何影响**。

为什么要这样初始化?

1. **保证初始行为不变**

由于（B）被初始化为零，模型在训练开始时的前向输出与原始预训练模型完全一致，避免了因新增参数而引入的性能波动。

1. **渐进式学习新任务**

随着训练进行，梯度会逐步更新（B）和（A），模型才开始在低秩子空间中学习适配新任务的方向，实现“从零开始”的平滑微调。

1. **避免训练不稳定**

如果（A）和（B）同时随机初始化，可能在一开始就对输出产生较大扰动，影响训练稳定性；当前初始化方式有效降低了这种风险。

4、训练和推理阶段

训练时：

- 冻结原模型参数 W；
- 只训练新增的低秩矩阵 A 和 B；
- 推理时，可将 ΔW 合并回 W，不增加额外计算。

因此，LoRA 既节省显存，又保持推理效率。

5、loar应该作用在哪?

给定参数上限时，应该优先将LoRA应用到哪里?

	# of Trainable Parameters = 18M						
Weight Type	W_q	W_k	W_v	W_o	W_q, W_k	W_q, W_v	W_q, W_k, W_v, W_o
Rank r	8	8	8	8	4	4	2
WikiSQL ($\pm 0.5\%$)	70.4	70.0	73.0	73.2	71.4	**73.7**	**73.7**
MultiNLI ($\pm 0.1\%$)	91.0	90.8	91.0	91.3	91.3	91.3	**91.7**

可以看到，应该尽可能的在更多参数矩阵应用LoRA，即使r很小，而不是弄一个大的r在一个参数矩阵上。

6、r值取值?

	Weight Type	$r=1$	$r=2$	$r=4$	$r=8$	$r=64$
WikiSQL($\pm 0.5\%$)	W_q	68.8	69.6	70.5	70.4	70.0
	W_q, W_v	73.4	73.3	73.7	73.8	73.5
	W_q, W_k, W_v, W_o	74.1	73.7	74.0	74.0	73.9
MultiNLI ($\pm 0.1\%$)	W_q	90.7	90.9	91.1	90.7	90.7
	W_q, W_v	91.3	91.4	91.3	91.6	91.4
	W_q, W_k, W_v, W_o	91.2	91.7	91.7	91.5	91.4

实验证明LoRA在r很小时准确率依然很高（如果下游任务和预训练任务相差过大，则应适当增大r）

1.5、提示工程

> Ref: https://www.promptingguide.ai/zh

时至今日，改进提示词显然有助于在不同任务上获得更好的结果。这就是提示工程背后的整个理念。你可以通过简单的提示词（Prompts）获得大量结果，但结果的质量与你提供的信息数量和完善度有关。一个提示词可以包含你传递到模型的*指令*或*问题*等信息，也可以包含其他详细信息，如*上下文*、*输入*或*示例*等。你可以通过这些元素来更好地指导模型，并因此获得更好的结果。

1.5.1、提示工程的定义

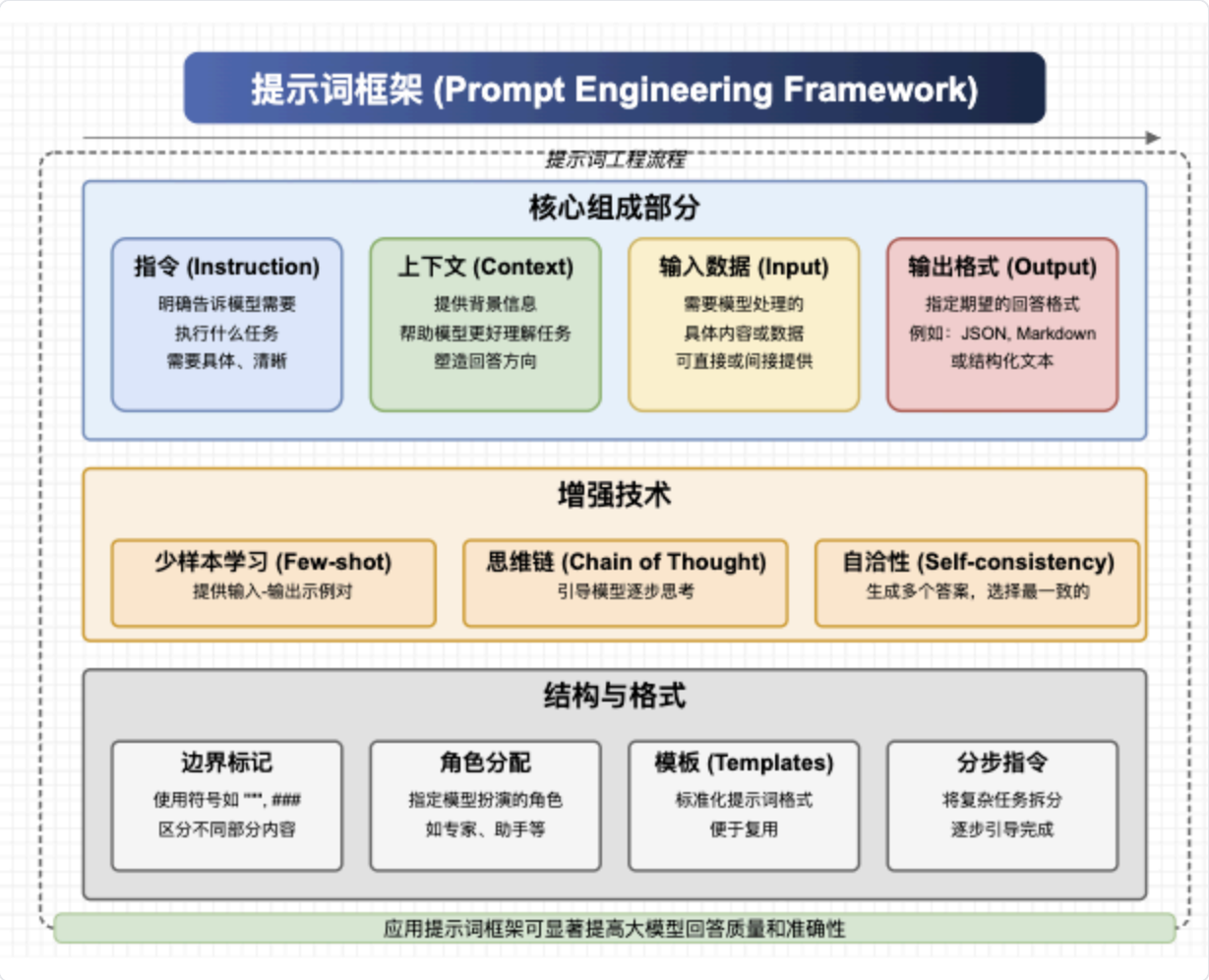
提示词是短小的文本片段，为大模型提供额外的信息或指导，例如模型生成文本的风格。提示词包括两种:系统提示词和用户提示词。

- **系统提示词**：在向大模型提出问题或任务前，向大模型提供上下文、说明和指南的一种方式。通过系统提示词，用户可以指定大模型的角色、个性、语气等信息，以帮助大模型更好地理解和响应用户的输入。例如：“你是一个知识渊博的学者，回答问题要严谨且富有条理。“
- **用户提示词**：用户在与大模型交互时输入的具体指令、问题、请求等。它是用户基于自身需求和目的而向大模型传达的具体信息，希望大模型根据这个提示来生成相应的输出。

提示工程就是通过为大模型提供更多、更优质的上下文信息，从而提升大模型在任务上表现的过程。通过完善提示词，让大模型充分理解任务，从而输出更优质的结果。

1.5.2、通用提示模板

下图给出了提示词应该考虑的要素,展示了一个完整的提示词工程框架，用于系统化地设计和优化与大模型的交互方式。框架从核心组成部分出发，强调指令、上下文、输入数据与输出格式四个要素的协同作用，确保模型明确任务目标、理解背景信息、获取必要数据并按预期形式输出结果。在此基础上，引入少样本学习、思维链和自洽性等增强技术，以提升模型推理能力与答案稳定性。同时，通过边界标记、角色分配、模板化和分步指令等结构与格式设计，使提示更加规范、清晰和可复用。



这里给出了一个通用的提示词模板，包含角色，背景，目标，约束，工作流，外部文档，输出规则等。

```
prompt='''
# 任务指令
作为一位[专业角色]，请[执行具体任务]。

# 背景上下文
以下是关于[主题/问题]的背景信息：
- [相关信息点1]
```

```
- [相关信息点2]
- [相关信息点3]

# 输入数据
"""
[这里放置需要处理的文本/数据/内容]
"""

# 要求与约束
请遵循以下要求：
1. [要求1，例如：分析的深度]
2. [要求2，例如：考虑特定因素]
3. [要求3，例如：使用特定方法]
4. [限制/约束，例如：回答字数限制]

# 输出格式
请按照以下结构提供您的回答：

## [第一部分标题]
[对第一部分内容的描述要求]

## [第二部分标题]
[对第二部分内容的描述要求]

## [第三部分标题]
[对第三部分内容的描述要求]

# 示例（少样本学习）
输入：[示例输入]
输出：[示例期望输出]

# 思考过程
在提供最终答案前，请先：
1. [思考步骤1]
2. [思考步骤2]
3. [思考步骤3]
4. 对您的回答进行自我评估和审查

# 按照一下JSON格式进行输出
{
```

```
"思考过程":
"输出结果":
}
'''
```

1.5.3、提示工程技巧

1、零样本提示（Zero-Shot）

如今，经过大量数据训练并调整指令的LLM能够执行零样本任务。以下是我们使用的一个示例：

提示：

```
将文本分类为中性、负面或正面。文本：我认为这次假期还可以。情感：
```

输出：

```
中性
```

请注意，在上面的提示中，我们没有向模型提供任何示例——这就是零样本能力的作用。

2、Few-shot 少样本学习

> **参考论文**：https://arxiv.org/pdf/2302.13971、https://arxiv.org/pdf/2109.01652

虽然大型语言模型展示了惊人的零样本能力，但在使用零样本设置时，它们在更复杂的任务上仍然表现不佳。少样本提示可以作为一种技术，以启用上下文学习，我们在提示中提供演示以引导模型实现更好的性能。演示作为后续示例的条件，我们希望模型生成响应。

通过提供几个输入-输出示例对，帮助模型理解任务期望。例如：

> 请分析以下文本的情感倾向，判断为"正面"、"负面"或"中性"。
>
> 示例1:
>
> 文本: "这家餐厅的服务太差了，等了一个小时才上菜，而且味道也不怎么样。"
>
> 情感: 负面

示例2:

文本: "新手机运行速度非常快，摄像功能也很强大，非常满意这次购买。"

情感: 正面

现在请分析:

文本: "虽然价格有点贵，但考虑到产品质量和售后服务，我认为还是很值得的。"

情感:

3、提供CoT思考过程

> **参考论文**：https://arxiv.org/pdf/2201.11903

思维链（**Chain-of-Thought， CoT)提示**是一种让语言模型在给出最终答案之前，先生成一系列中间推理步骤的方法。与直接输出答案的标准提示相比，CoT 显式引导模型“像人类一样逐步思考”，从而提升其在复杂推理任务（如数学计算、逻辑推理、常识推断等）中的准确率与可解释性。

该方法最早由 Wei等人（2022）提出，发现当模型规模足够大时，通过展示中间推理过程，模型能够表现出显著增强的推理能力，这被视为一种“新兴能力”。例子如下：

> 📚 **标准提示（Standard Prompting+Few-shot)**
>
> **模型输入（Model Input)**
>
> 问：罗杰有5个网球。他又买了2罐网球。每罐有3个网球。他现在一共有多少个网球?
>
> 答：答案是11。
>
> 问：自助餐厅有23个苹果。如果他们用了20个做午餐，又买了6个，现在还有多少个苹果?
>
> **模型输出（Model Output)**
>
> 答：答案是27。 ❌
>
> **思维链提示（Chain-of-Thought Prompting)**
>
> **模型输入（Model Input)**
>
> 问：罗杰有5个网球。他又买了2罐网球。每罐有3个网球。他现在一共有多少个网球?
>
> 答：罗杰一开始有5个球。2罐、每罐3个球，一共是6个球。5 + 6 = 11。答案是11。

问：自助餐厅有23个苹果。如果他们用了20个做午餐，又买了6个，现在还有多少个苹果?

模型输出（Model Output）

答：自助餐厅原来有23个苹果。他们用了20个做午餐，所以剩下23 − 20 = 3个。又买了6个苹果，所以现在有3 + 6 = 9个。答案是9。 ✅

我们可以看到在提供推理步骤时得到了完美的结果。作者声称这是足够大的语言模型才会出现的新兴能力。

- **让我们逐步思考。**

最近提出的一个新想法是 零样本CoT（ Kojima 等人，2022年），它基本上涉及将“让我们逐步思考”添加到原始提示中。让我们尝试一个简单的问题，看看模型的表现如何：

📌 **模型输入（Model Input）**

问：我去市场买了10个苹果。我给了邻居2个苹果和修理工2个苹果。然后我去买了5个苹果并吃了1个。我还剩下多少苹果?

模型输出（Model Output）

答：11个苹果

模型输入（Model Input）

问：我去市场买了10个苹果。我给了邻居2个苹果和修理工2个苹果。然后我去买了5个苹果并吃了1个。我还剩下多少苹果?**让我们逐步思考。**

模型输出（Model Output）

答：首先，您从10个苹果开始。您给了邻居和修理工各2个苹果，所以您还剩下6个苹果。然后您买了5个苹果，所以现在您有11个苹果。最后，您吃了1个苹果，所以您还剩下10个苹果。

4、自洽性

参考论文：https://arxiv.org/pdf/2203.11171

由 Wang等人（2022） 提出，自我一致性旨在“替换链式思维提示中使用的天真贪婪解码方法”。其想法是通过少样本 CoT 采样多个不同的推理路径，并使用生成结果选择最一致的答案。

这有助于提高 CoT 提示在涉及算术和常识推理的任务中的性能。自洽性（Self-Consistency）是一种通过**多次生成不同推理路径，再进行投票或汇总**来提高答案稳定性与正确率的提示方法，常用于复杂推理、数学、逻辑和规划任务中。

Prompt 1（普通提问）：

当我6岁时，我的妹妹是我的一半年龄。现在我70岁了，我的妹妹多大?

模型输出：

她现在是 35 岁。（❌ 错误）

Prompt 2（自洽性提示）：

请用多种不同的推理路径独立思考这个问题，并在最后给出你认为最可靠的答案。

问题是：当我6岁时，我的妹妹是我的一半年龄。现在我70岁了，我的妹妹多大?

模型输出：

思路一：6岁时妹妹是3岁，年龄差为3岁，现在我70岁，妹妹70 − 3 = 67岁。

思路二：年龄差始终不变，为3岁，现在妹妹是70 − 3 = 67岁。

思路三：当 6 岁时，妹妹 3 岁，比例是 1/2，因此现在妹妹是70 / 2 = 35 岁

综合多个推理结果，最终答案为：67岁。

5、其他技巧

1. **使用确定语言**

- 必须使用“必须” “一定”等明确词语，避免“应该” “最好”等模糊表达。

1. **明确目标和对象**

- 指定受众、篇幅或输出效果。 - 例： - 坏：解释提示工程的概念。 - 好：使用 2-3 句话向高中学生解释提示工程，让他能快速理解。

1. **指定任务步骤**

- 如果任务有明确流程，必须在 Prompt 中列出完成步骤。 - 例： - 坏：写一篇关于气候变化的文章。 - 好：为了写文章，请完成以下步骤： 1. 定义气候变化及其原因。 2. 描述气候变化对环境的影响。 3. 讨论社会和经济的应对措施。 4. 提出解决策略。

1. **提供充分上下文**

- 包括角色设定、场景信息、相关文档或参考资料。 - 例： - 你是人工智能专家，公司想让员工了解 AI 基础知识。 - 请解释 AI 的定义、主要组成部分及其协作方式。

1. **正向引导**

- 聚焦模型应做的事情，而非限制模型行为。 - 例： - 坏：如果没有电影推荐，不要胡乱推荐。 - 好：如果没有电影推荐，必须回答“抱歉，今天找不到电影推荐”。

1. **控制任务复杂度**

- 单一任务不要超过 3 个步骤，复杂任务需拆分为子任务。

1. **使用模块化标记**

- 使用 `##`、 **等标记区分不同模块，提高可读性。**

1. 持续优化

- 收集模型不当输出，分析问题，重新调整 Prompt。

二、基于LoRA微调LLM项目实践

2.1、项目简介

项目简介：**本项目以多语言电商评论文本为输入，围绕评论自动摘要生成任务，构建了一个基于大语言模型的文本生成系统。项目采用 LoRA（Low-Rank Adaptation）参数高效微调技术，在保持基础模型能力的同时，实现对多语言、多风格电商评论的高质量摘要生成。整体流程完整覆盖数据清洗与构建、指令式监督微调、LoRA 微调训练、推理部署及摘要效果评估等关键环节。模型通过引入BLEU 等主流自动评价指标，对生成摘要的内容覆盖度与语言质量进行定量分析，具备在电商智能客服、商品评价汇总等实际场景中的应用价值。**

学习收获：**通过本项目，系统掌握大语言模型在文本生成任务中的微调流程，理解指令数据构建方法与 LoRA 参数高效微调机制，完成一个面向多语言电商评论的自动摘要生成算法和模型。**

评价指标：BLUE-1、BLUE-2**

2.2、代码

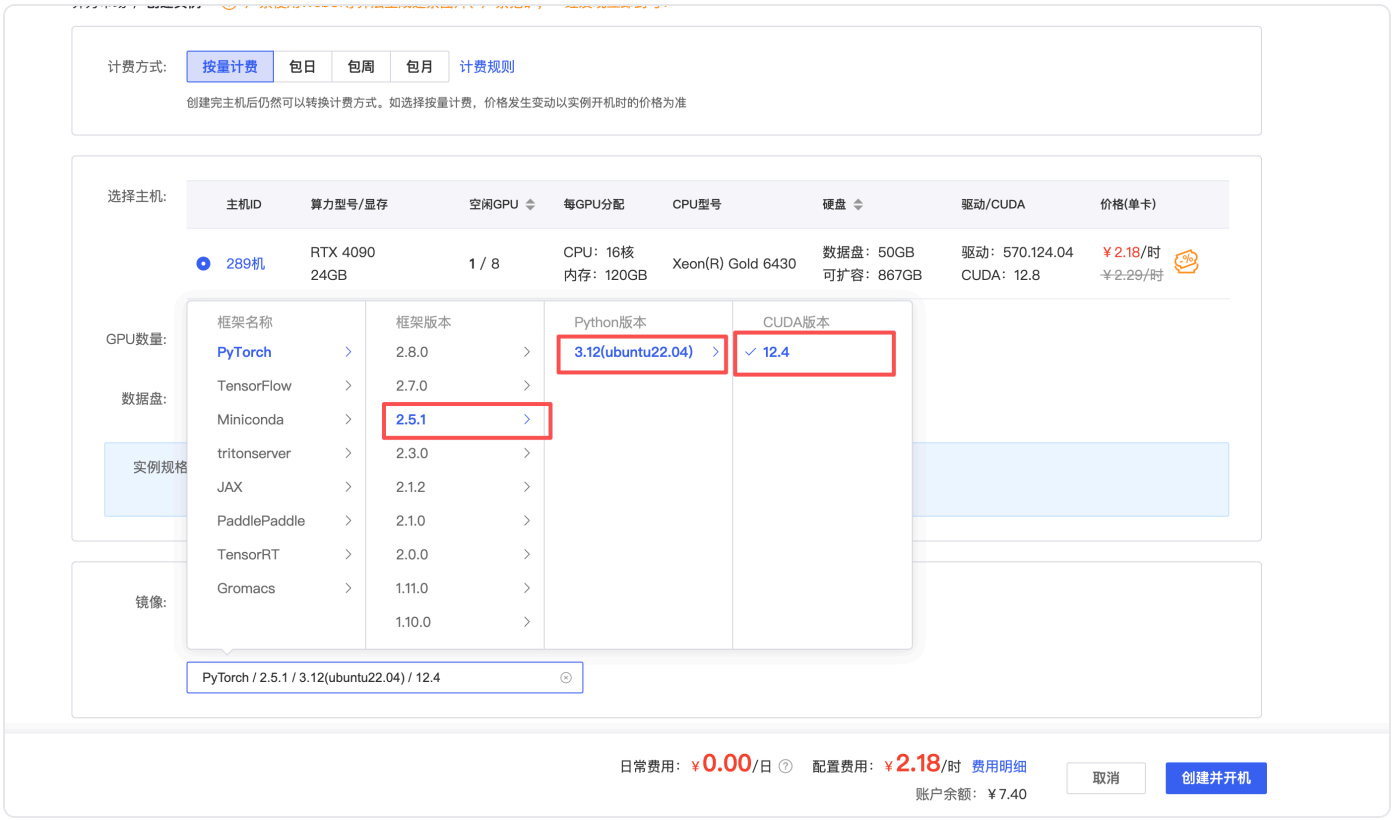
LLM（可下载文档）

此处为动图（内容见飞书原文）

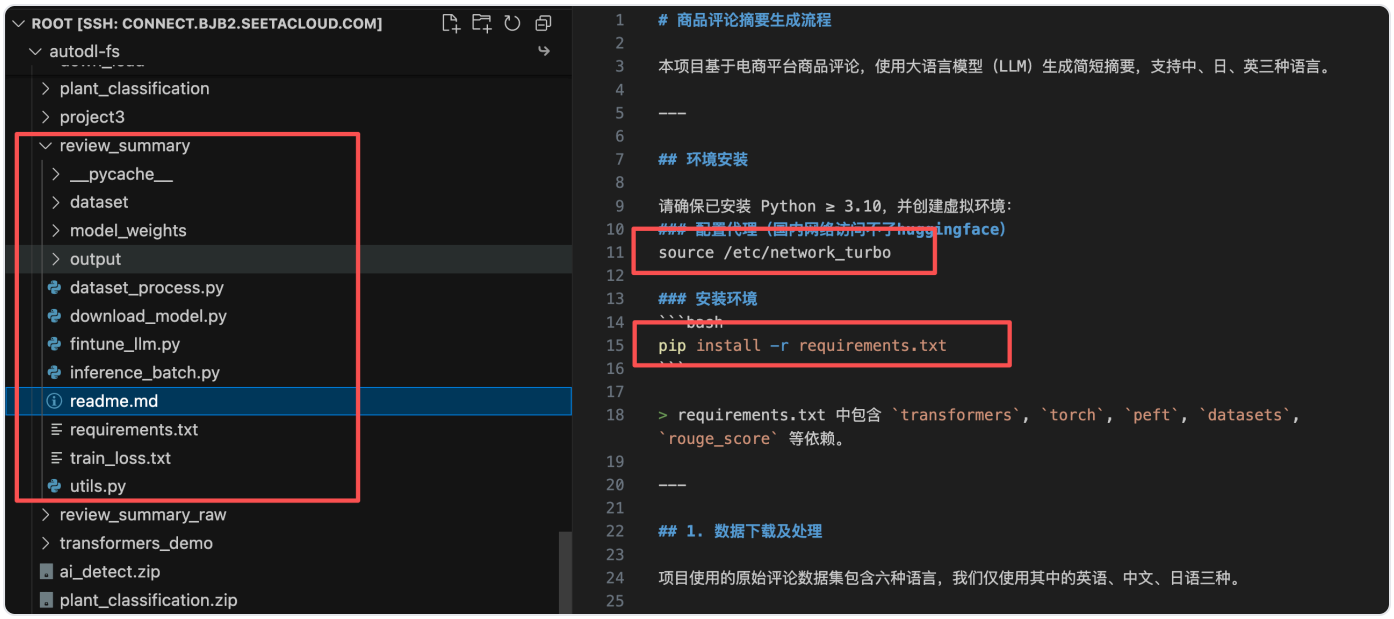
```
.
├── dataset/                # 存放原始和划分后的训练/验证/测试数据集
├── dataset_process.py      # 数据格式转换、切分等预处理脚本
├── download_model.py       # 下载预训练模型或权重文件的脚本
├── fintune_llm.py          # 大语言模型微调主程序（训练入口）
├── inference_batch.py      # 批量推理脚本
├── model_weights/          # 存放训练后或下载得到的模型权重
├── output/                 # 推理结果、日志或中间输出文件目录
├── requirements.txt        # 项目依赖的 Python 包列表
├── readme.md               # 项目说明文档与使用指南
├── train_loss.txt          # 训练损失的文本记录文件
├── utils.py                # 公共工具函数（如日志、评估、数据处理等）
```

- 上传代码文件到/root/autodl-fs 目录下

- 新建实例，注意和要上传的文件在一个区域，比如北京B区，选择4090。



- 配置环境



- 下载数据：

```
python3 dataset_process.py
```

> 下载网速可能不稳定，我将下载下来的数据存放到百度网盘：地址。其中，包含csv类型的（从官方上直接下载，对应代码前半段）和jsonl类型（我们处理之后的，对应代码后半段）。
>
> 大家可以只下载jsonl类型，然后新建一个 `dataset` 文件夹，放到里面，用于接下来LLM微调。

- 下载模型权重

注意先配置代理 `source /etc/network_turbo` ,然后下载权重，项目使用预训练模型 `Qwen/Qwen3-0.6B` ，方便快速学习，也可根据算力选择更大模型。

```
source /etc/network_turbo
python3 download_model.py
```

- 如果想直接看代码，就看代码中readme.md

2.3、数据集

2.3.1、数据集描述

- 来源：kaggle网站

项目使用的原始评论数据集包含六种语言，我们仅使用其中的英语、中文、日语三种。数据集来源：Amazon Reviews Multi-Language。 生成训练集、验证集和测试集：

数据集	样本数量
训练集	600,000
验证集	15,000
测试集	15,000

- 数据样例

- 原始数据

```
,review_id,product_id,reviewer_id,stars,review_body,review_title,language,product_category
0,de_0591095,product_de_0301773,reviewer_de_0787123,1,"Das Produkt kam bis heute nicht bei mir
1,de_0987834,product_de_0720578,reviewer_de_0983846,1,"Gebrauchte Spinner, teilzerlegt und teil
```

- 通过 `dataset_process.py` 脚本转换为，处理之后的

```
{"review_body": "Great price. Good quality.", "review_title": "Five Stars"}
{"review_body": "Fits my 2016 F150 XLT great. Now I dont have to listen to it hitting side to s
{"review_body": "Just what I needed! Perfect for western theme party.", "review_title": "Perfec
{"review_body": "真的很好，孩子学到很多东西，也改到一下不好的习惯", "review_title": "励志书"}
{"review_body": "手表刚收到，还不错，只是外包装真的很烂，就一个很薄的纸盒子，还开着口，说明书全英
{"review_body": "很好，秒杀价格优惠。质量不错，使用方便安全。满意。", "review_title": "很好用的指
{"review_body": "侦探故事，哪个小孩子不喜欢？价格合理，服务也棒，赞！", "review_title": "好"}
{"review_body": "購入、貼付け後2週間もたたないうちに、上側から剥がれてきて画面から浮いた状態にな
{"review_body": "以下の3点の理由により、期待はずれの粗悪品。 ①他の方のレビューにもある通り、天板
{"review_body": "この商品の内容等確認した購入したのですが、そのとおりなかなか設定ができなく、知人
{"review_body": "テストした結果、4000mahのスマホ一回と30％分、スマホのバッテリー残量の表示が正し
```

- `review_body`： 用户评论正文
- `review_title`： 评论标题

📌 示例数据

中文示例

> **原始评论（输入）**

> 收到货这个样子 上面都是锈 擦了擦 缝隙地方擦不掉 就这样吧 这价位戴着玩玩 懒得退

> **摘要（输出）**

> 锈迹斑驳

英文示例

> **原始评论（输入）**

> Sadly have to give it 2 stars. It’ s such a cute outfit but too big.

> My son wears 12-18… but the size was huge… the shirt was pinkish.

> **摘要（输出）**

> Too big

2.3.2、数据集预处理

0、 Datasets

`datasets` 是 Hugging Face 提供的一个 Python 库，用于加载、组织、处理和共享结构化数据集，主要服务于自然语言处理及相关机器学习任务。该库通过统一的数据表示方式，将不同来源、不同格式的数据转化为标准化的 `Dataset` 对象，使数据加载、预处理与模型训练过程具备可复现性与工程稳定性。

`1、load_dataset` 函数（属于 `datasets` 库）

> load_dataset link: https://huggingface.co/docs/datasets/loading#json

`load_dataset` 是 `datasets` 库中的接口，用于从本地文件系统或 Hugging Face 数据仓库中加载数据，并自动解析为统一的数据结构。该函数屏蔽了底层文件格式差异，使用户能够以一致的方式访问 JSON、CSV、文本或远程数据集。以读取本地json文件为例：

```
raw_dataset = load_dataset('json', data_files=file_path)['train']
```

该调用完成以下操作：

1. 解析指定路径下的 JSON 文件；

2. 构建标准化的 `DatasetDict` 对象；
3. 提取其中的训练集划分，得到单一 `Dataset` 实例，作为后续处理的输入。

JSON

JSON 文件可以直接使用load_dataset() 函数加载，如下所示：

```
>>> from datasets import load_dataset
 >>> dataset = load_dataset( "json" , data_files= "my_file.json" )
```

JSON 文件格式多种多样，但我们认为最有效的格式是使用多个 JSON 对象；每一行代表一行数据。例如：

```
{ "a" : 1 , "b" : 2.0 , "c" : "foo" , "d" : false }
{ "a" : 4 , "b" : -5.5 , "c" : null , "d" : true }
```

2、`dataset.map` 方法（属于 `Dataset` 对象）

`map` 是 `Dataset` 对象提供的变换接口，用于对数据集中的样本进行函数式映射。其本质是对每一个样本（或批次样本）执行同一转换逻辑，并返回结构一致的新数据集。在自然语言处理任务中，`map` 通常用于完成以下工作：

- 文本分词与编码；
- 标签映射与格式规范化；
- 特征构造与字段重命名。

```
dataset.map(lambda examples: tokenize_function(tokenizer, examples))
```

该调用将原始文本样本转换为模型可接受的张量格式（如 `input_ids`、`attention_mask`），并生成新的数据字段。

3、数据预处理函数的组织方式

```
from datasets import load_dataset
def preprocess_data(file_path, sample_size=None, DEFAULT_SEED=42):
```

```
    raw_dataset = load_dataset('json', data_files=file_path)['train']
    '''
    Dataset({
        features: ['review_body', 'review_title'],
        num_rows: 15000
    })
    (Pdb) raw_dataset[0]
    {'review_body': 'These are AWFUL. They are see through, the fabric feels like tablecloth, a
    '''
    # 数据采样与随机化
    if sample_size:
        raw_dataset = raw_dataset.shuffle(seed=DEFAULT_SEED).select(range(sample_size))
    return raw_dataset
```

该函数将数据加载逻辑封装为独立模块，其目的在于：

- 统一数据入口；
- 提高代码可读性与可维护性；
- 为后续扩展采样策略或数据增强策略预留接口。

4、训练前的数据处理流程

- 这部分代码在fintune_llm.py文件中，其中 `generate_review_title_prompt_tmp0` 和 `apply_chat_template` 函数在utils.py中，在fintune_llm.py中通过import * 导入的。

```
def apply_chat_template(system, user):

    prompt =  f"<|im_start|>system\n{system}<|im_end|>\n<|im_start|>user\n{user}<|im_end|>\n<|i
    return prompt

def generate_review_title_prompt_tmp0(review_body):
    system_prompt = (
        "你将看到一个来自电商平台的商品评论，请根据评论内容，生成简短总结。"
    )
    user_prompt = review_body

    # 使用 apply_chat_template 格式
```

```
    return apply_chat_template(system_prompt, user_prompt)

def tokenize_function(tokenizer, examples):
    """
    将文本数据转换为模型所需的输入格式

    处理逻辑:
    1. 构建系统提示和用户指令
    2. 将标签转换为文本形式
    3. 拼接指令和响应生成完整序列
    4. 处理长度限制和特殊token

    参数:
        examples (dict): 包含'review_body'和'review_title'的样本字典
        例子： {'review_body': '刚收到货，烧水时有嗡嗡的声音，不知道是不是产品缺陷。', 'review_t

    返回:
        dict: 包含input_ids, attention_mask, labels的字典
    """
    MAX_LENGTH = 1024  # 模型最大上下文长度

    # ref https://github.com/QwenLM/Qwen/blob/main/finetune.py
    # 构建指令部分
    instruction = tokenizer(
        generate_review_title_prompt_tmp0(examples['review_body']),
        add_special_tokens=False  # 禁用自动添加特殊token
    )
    '''
    generate_review_title_prompt_tmp0(examples['review_body'])->

    (Pdb) apply_chat_template(system_prompt, examples['review_body'])
    '<|im_start|>system\n你将看到一个来自电商平台的商品评论，请根据评论内容，生成简短总结。<|im_

    (Pdb) instruction
    {'input_ids': [151644, 8948, 198, 56568, 44063, 101038, 46944, 101919, 110514, 109348, 8564
    '''
    # 构建响应部分（真实标签）
    response = tokenizer(
        f"{examples['review_title']}",
        add_special_tokens=False
```

```
    )
    '''
    (Pdb) response
    {'input_ids': [100228, 52510, 13343, 104351, 104590, 117377, 117377, 104668], 'attention_ma
    '''
    # 将instructio部分和response部分的input_ids拼接，并在末尾添加eos token作为标记结束的token
    input_ids = instruction["input_ids"] + response["input_ids"] + [tokenizer.pad_token_id]
    '''
    (Pdb) input_ids
    [151644, 8948, 198, 56568, 44063, 101038, 46944, 101919, 110514, 109348, 85641, 11, 220, 14
    '''
    # 注意力掩码，表示模型需要关注的位置
    attention_mask = instruction["attention_mask"] + response["attention_mask"] + [1]
    '''
    (Pdb) attention_mask
    [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
    '''
    # 对于instruction，使用-100表示这些位置不计算loss（即模型不需要预测这部分）
    labels = [-100]*len(instruction["input_ids"]) + response["input_ids"] + [tokenizer.pad_toke
    '''
    (Pdb) labels
    [-100, -100, -100, -100, -100, -100, -100, -100, -100, -100, -100, -100, -100, -100, -100,
    [-100, -100, -100,  -100, 100228, 52510, 13343]
    input_ids[:4] label [5]
    input_ids[:5] label [6]
    input_ids[:6] label [7]



    '''
    # 长度截断处理
    if len(input_ids) > MAX_LENGTH:
        input_ids = input_ids[:MAX_LENGTH]
        attention_mask = attention_mask[:MAX_LENGTH]
        labels = labels[:MAX_LENGTH]

    # import pdb;pdb.set_trace()
    return {
        "input_ids": input_ids,
        "attention_mask": attention_mask,
        "labels": labels
```

```
    }
# 加载数据
dataset = preprocess_data(args.data_path, sample_size=args.sample_size)
# 应用分词处理
tokenized_dataset = dataset.map(lambda examples: tokenize_function(tokenizer, examples), remove
```

该流程完成两个关键阶段：

1. **数据读取阶段**：将原始文件转换为标准化数据集对象；
2. **特征构造阶段**：将原始文本字段转换为模型输入特征，并删除不再需要的原始字段。

`remove_columns=dataset.column_names` 的作用是移除原始文本与中间字段，仅保留模型训练所需的数值化输入。

2.4、模型

2.4.1、下载

```
from modelscope import snapshot_download
# #模型下载
model_dir = snapshot_download('Qwen/Qwen3-0.6B', cache_dir='./model_weights', revision='master'
```

项目使用预训练模型 `Qwen/Qwen3-0.6B`，方便快速学习，也可根据算力选择更大模型。模型文件结构示例：

```
├── LICENSE                # 开源许可协议，说明模型的使用、修改和分发权限
├── README.md              # 项目说明文档，包含模型介绍、使用方法和示例
├── config.json            # 模型结构配置文件，定义层数、隐藏维度、注意力头数等网络参数
├── configuration.json     # 模型加载兼容配置或框架扩展配置（部分仓库用于适配不同工具链）
├── generation_config.json # 文本生成参数配置，如 max_new_tokens、temperature、top_p 等
├── merges.txt             # BPE 分词的子词合并规则文件
├── model.safetensors      # 模型权重文件（约 1.5GB），存储所有训练得到的参数
├── tokenizer.json         # 分词器完整定义文件，包含词表、规则及预处理方式
```

```
├── tokenizer_config.json   # 分词器行为配置，如特殊 token、大小写处理等
└── vocab.json              # 词表文件，将每个 token 映射为唯一的整数 ID
```

2.4.2、加载

```
from transformers import AutoTokenizer, AutoModelForCausalLM
from peft import LoraConfig, TaskType, get_peft_model

def initialize_model(model_path: str, use_lora: bool = True):
    # 初始化分词器
    tokenizer = AutoTokenizer.from_pretrained(
        model_path,
        use_fast=False,
        trust_remote_code=True
    )

    # 加载基础模型
    model = AutoModelForCausalLM.from_pretrained(
        model_path,
        torch_dtype=torch.bfloat16,  # 使用bfloat16混合精度
        device_map="auto"            # 自动设备分配
    )

    # 配置LoRA适配器
    if use_lora:
        lora_config = LoraConfig(
            task_type=TaskType.CAUSAL_LM,
            target_modules=[  # 覆盖关键注意力模块
                "q_proj", "k_proj", "v_proj", "o_proj",
                "gate_proj", "up_proj", "down_proj"
            ],
            inference_mode=False,
            r=8,                  # LoRA秩（平衡效果与效率）
            lora_alpha=32,         # 缩放因子（一般=2r 或 4r）
            lora_dropout=0.1,       # 正则化比率
        )
        model = get_peft_model(model, lora_config)
```

```
    # 启用梯度检查点（显存优化）
    model.enable_input_require_grads()
    model.gradient_checkpointing_enable()

    print(model.print_trainable_parameters()
)
    return model, tokenizer
```

- lora_alpha什么意思?

`lora_alpha = 32` 表示将低秩更新项按 $\frac{\alpha}{r} = \frac{32}{8} = 4$ 倍比例缩放，使 LoRA 适配器对模型参数更新具有足够影响力。结合权重更新公式：

$$W' = W + \frac{\alpha}{r} \cdot BA,$$

可得当前配置下：

$$W' = W + 4 \cdot BA.$$

其中，W 为原始冻结权重，BA 为低秩更新矩阵。

“通常取 (2r) 或 (4r)”表示这是在实践中常用的经验设置范围，用于平衡训练稳定性与适配能力。

2.5、损失函数和优化器

- 损失函数

在 HuggingFace 训练框架中，损失函数通常由模型自身在前向传播（`forward()`）阶段自动计算，而不是在 `Trainer` 中手动指定。只要输入数据中包含 `labels` 字段，模型就会在内部调用交叉熵损失函数（`CrossEntropyLoss`），并将计算得到的损失值作为输出的一部分返回。`Trainer` 在训练过程中会自动读取该 `loss`，并据此执行反向传播和参数更新，从而完成整个训练流程。这种设计使训练流程更加简洁，同时保证不同模型能够统一地使用其内置损失计算机制。

- 优化器

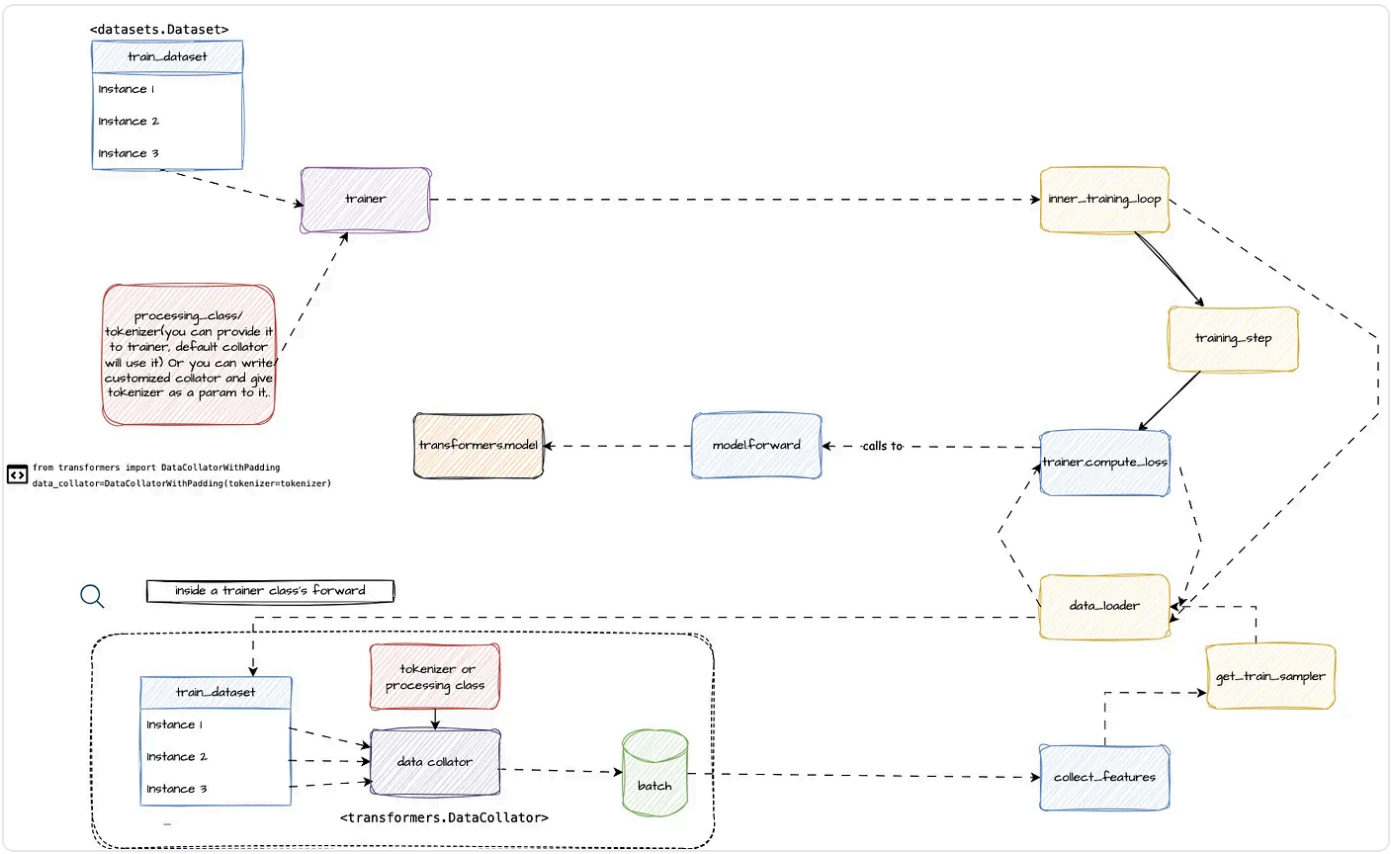
```
    # 训练参数配置
    training_args = TrainingArguments(
```

```
        output_dir=args.output_dir,
        per_device_train_batch_size=args.batch_size,       # 根据显存调整
        gradient_accumulation_steps=args.gradient_accumulation_steps,  # 梯度累积（模拟更大batc
        logging_steps=args.logging_steps,                  # 监控训练状态
        num_train_epochs=args.epochs,                      # 训练轮数
        save_steps=args.save_steps,                        # 检查点间隔
        learning_rate=args.learning_rate,                  # 学习率（LoRA典型值）
        save_on_each_node=True,                            # 分布式训练设置
        gradient_checkpointing=True,                       # 启用梯度检查点
        bf16=True,                                         # 使用bfloat16混合精度训练
        remove_unused_columns=False,                       # 保留未使用列（用于标签处理）
        optim="adamw_torch",                               # 优化器类型（AdamW）
        report_to="none",
        logging_dir="./logs"
    )

    # 初始化训练器
    trainer = Trainer(
        model=model,
        args=training_args,
        train_dataset=dataset,
        data_collator=DataCollatorForSeq2Seq(tokenizer=tokenizer, padding=True)
    )
```

2.6、训练

> Ref： https://huggingface.co/learn/llm-course/zh-CN/chapter3/3




```
# 启动训练流程
trainer.train()
```

调用 `trainer.train()` 会启动整个训练流程，这一行代码封装了模型训练的核心环节，从数据加载到参数更新，可分为六大步骤。

第一步是**训练环境初始化**。Trainer 会根据 `TrainingArguments` 中的配置加载训练参数，包括每个设备的批量大小、梯度累积步数、训练轮数、学习率、优化器类型、学习率调度策略、混合精度设置、日志记录间隔以及检查点保存策略。

第二步是**数据准备**。Trainer 会根据指定的训练数据集和 `DataCollator` 构建训练数据迭代器。数据迭代器会**按照批次将样本整理成模型可接受的格式**，例如 `input_ids`、`attention_mask` 和 `labels`。在构建过程中，还会处理 padding、shuffle、batch 对齐以及 GPU 内存优化等操作，保证训练效率和稳定性。

第三步是**前向传播与损失计算**。在每个训练步骤中，Trainer 会将一个 batch 的输入送入模型的 `forward()` 方法。模型根据输入计算输出 logits，同时如果输入包含 `labels`，模型会自动在内部调用交叉熵损失函数计算 loss。

第四步是**反向传播与梯度处理**。Trainer 会调用 `loss.backward()` 将损失梯度累积到模型参数上。如果启用了梯度累积，Trainer 会在累积指定步数后再进行参数更新。

第五步是**优化器更新与学习率调整**。在完成梯度累积后，Trainer 会调用优化器 `step()` 更新模型参数，同时使用学习率调度器 `scheduler.step()` 调整当前学习率。之后，梯度会被清零，为下一步训练做准备。

第六步是**训练监控、日志与检查点保存**。Trainer 会按照配置定期输出训练日志，包括当前 loss、学习率、训练进度等指标，以便分析训练状态。同时，Trainer 会定期保存模型检查点，包括模型权重、优化器状态和调度器状态，使训练可以在中断后恢复。训练完成后，Trainer 会保存最终模型，并返回训练状态对象，其中包含总训练步数、平均损失等信息。

总结，一行 `trainer.train()` 实际上封装了完整的训练循环，包括环境初始化、数据加载、前向传播、自动损失计算、反向传播、梯度累积、优化器更新、学习率调度、日志记录、检查点保存以及训练结束处理。

2.6.1、训练参数

```
 parser = argparse.ArgumentParser(description="Fine-tune LLM with LoRA")
parser.add_argument("--model_path", type=str, default="./model_weights/Qwen/Qwen3-0.6B", help="
parser.add_argument("--data_path", type=str, default="./dataset/train.jsonl", help="训练数据路径
parser.add_argument("--output_dir", type=str, default="./output", help="模型保存路径")
parser.add_argument("--batch_size", type=int, default=8, help="每个设备的训练batch大小")
parser.add_argument("--gradient_accumulation_steps", type=int, default=1, help="梯度累积步数")
parser.add_argument("--logging_steps", type=int, default=10, help="日志记录步数")
parser.add_argument("--epochs", type=int, default=3, help="训练轮数")
parser.add_argument("--save_steps", type=int, default=100, help="保存检查点间隔")
parser.add_argument("--learning_rate", type=float, default=1e-4, help="学习率")
parser.add_argument("--sample_size", type=int, default=None, help="数据采样大小，可选")
parser.add_argument("--use_lora", type=bool, default=True, help="是否启用LoRA微调")
args = parser.parse_args()
```

参数	类型	默认值	说明
`--model_path`	str	`./model_weights/Qwen/Qwen3-0.6B`	预训练模型路径

`--data_path`	str	`./dataset/train.jsonl`	训练数据路径
`--output_dir`	str	`./output`	模型保存路径
`--batch_size`	int	12	每个设备训练的batch大小
`--gradient_accumulation_steps`	int	4	梯度累积步数，模拟更大batch
`--logging_steps`	int	10	日志记录步数，用于监控训练状态
`--epochs`	int	3	训练轮数
`--save_steps`	int	100	保存检查点间隔
`--learning_rate`	float	1e-4	学习率
`--sample_size`	int	60000	数据采样大小，可选
`--use_lora`	bool	True	是否启用LoRA微调

2.6.2、启动训练

```
python3 fintune_llm.py \
    --model_path ./model_weights/Qwen/Qwen3-0.6B \
    --data_path ./dataset/train.jsonl \
    --output_dir ./output \
    --batch_size 12 \
    --gradient_accumulation_steps 4 \
    --logging_steps 10 \
    --epochs 3 \
    --save_steps 100 \
    --learning_rate 1e-4 \
```

```
    --sample_size 60000 \
    --use_lora True
```

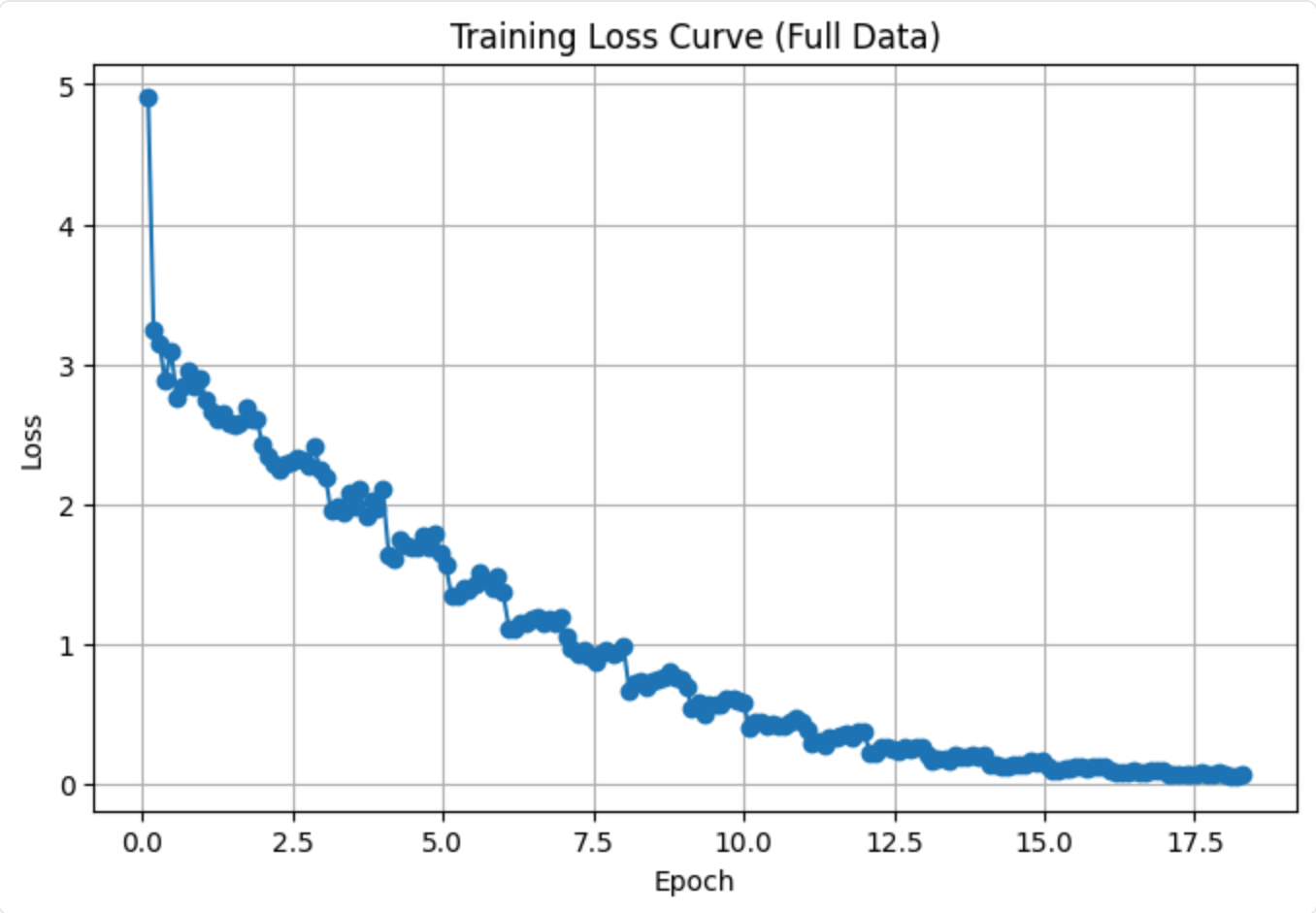
- 机器状态及训练速度：

- 显卡: RTX 4090（一共24GB) - 显存占用: 14 GB（`nvidia-smi` 命令可以获取显存状态，如下） - 训练速度: 1.1 step/s - 五分钟训练步数: \~300 steps - Loss 收敛: 约 2000 steps 基本收敛，到0.0587左右

```
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 580.105.08             Driver Version: 580.105.08     CUDA Version: 13.0     |
+-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================|
|   0  NVIDIA GeForce RTX 4090        On  |   00000000:CE:00.0 Off |                  Off |
|  0%   36C    P2            194W /  450W |   13957MiB /  24564MiB |     67%      Default |
|                                         |                        |                  N/A |
+-----------------------------------------+------------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI              PID   Type   Process name                        GPU Memory |
|        ID   ID                                                               Usage      |
|=========================================================================================|
|    0   N/A  N/A            3344      C   python3                               13948MiB |
+-----------------------------------------------------------------------------------------+
```

- 训练说损失函数



2.7、评价指标：

BLEU-1 与 BLEU-2 评价指标（含长度惩罚项）

2.7.1、指标定义

双语评估替补值（Bilingual Evaluation Understudy， BLEU）是一种基于 n元语法（n-gram）精确率并结合长度惩罚机制的自动评价指标，用于衡量生成文本与真实文本之间的重合程度。在仅考虑低阶 n-gram 时，常用指标包括：

- **BLEU-1**：基于 1-gram 的匹配；
- **BLEU-2**：基于 1-gram 与 2-gram 的联合匹配。

其统一形式为： $BLEU\text{-}n = BP \times \exp\left(\frac{1}{n}\sum_{k=1}^{n}\log p_k\right)$

其中：

- （p_k）：第 k 阶 n-gram 精确率；
- （BP）：长度惩罚项（Brevity Penalty)。

2.7.2、长度惩罚项

设：

- （c）：生成文本长度；
- （r）：真实文本长度，

则长度惩罚项定义为：

$$BP = \begin{cases} 1, & c > r \\ e^{(1-r/c)}, & c \leq r \end{cases}$$

该项用于防止生成文本过短但完全匹配，从而获得虚高的评价分数。

2.7.3、BLEU-1（基于词匹配）

（一）定义

BLEU-1 基于 1-gram 精确率（p_1）：在生成文本中，统计有多少个词能够在参考文本中找到对应匹配（每个词最多只能匹配参考文本中的出现次数），然后用这个匹配到的词数除以生成文本中的总词数，得到匹配比例。

最终：$BLEU\text{-}1 = BP \times p_1$

（二）示例

真实文本：我 喜欢 吃 苹果

生成文本：我 喜欢 吃

词命中数为 3，生成词总数为 3：

$$p_1 = \frac{3}{3} = 1$$

但生成文本长度（c = 3），真实文本长度（r = 4），故：

$$BP = e^{(1-4/3)} \approx 0.717$$

最终：

$$BLEU\text{-}1 = 0.717 \times 1 = 0.717$$

2.7.4、BLEU-2（基于词与词序匹配）

（一）定义

BLEU-2 在 BLEU-1 的基础上，同时考虑**单词级匹配（1-gram）和相邻词对匹配（2-gram）**。具体做法是：

先分别计算生成文本中，有多少单词与参考文本匹配，以及有多少相邻词对与参考文本匹配；

然后将这两个匹配比例进行平均融合（通过对数平均方式），再乘以长度惩罚项 BP，得到最终的 BLEU-2 分数。

其中，2-gram 精确率表示：在生成文本中，统计有多少个相邻词对能够在参考文本中找到对应匹配（每个词对最多匹配参考文本中的出现次数），再除以生成文本中的相邻词对总数。

（二）示例（同上文本）

生成 2-gram：

- 我 喜欢 ✓
- 喜欢 吃 ✓

真实 bigram：

- 我 喜欢
- 喜欢 吃
- 吃 苹果

则：

$$p_2 = \frac{2}{2} = 1, \quad p_1 = 1$$

长度惩罚项仍为：

$$BP \approx 0.717$$

最终：

$$BLEU\text{-}2 = 0.717 \times \exp\left(\frac{1}{2}(\log 1 + \log 1)\right) = 0.717$$

2.7.5、小结

- BLEU-1 和 BLEU-2 均由 **n-gram 精确率 + 长度惩罚项** 共同构成；
- 长度惩罚项在生成文本偏短时显著降低得分；
- 在单参考文本场景下，应特别注意 BP 对结果的影响。

2.8、模型推理

- 解码策略怎么作用于生成的过程中。

```
# https://huggingface.co/docs/transformers/main_classes/text_generation
generate_config = {
    "max_new_tokens": max_new_tokens,
    "min_new_tokens": min_new_tokens,
    "do_sample": do_sample,
    "top_p": top_p,
    "temperature": temperature,
    "repetition_penalty": repetition_penalty,
    # "pad_token_id": tokenizer.pad_token_id
}
```

```
    with torch.no_grad():
        outputs = model.generate(**inputs, **generate_config)
```

2.8.1、运行方式

```
python3 inference_batch.py \
    --input ./dataset/test.jsonl \
    --output ./output/inference_result.jsonl \
    --lora_path ./output/checkpoint-2000 \
    --model_name ./model_weights/Qwen/Qwen3-0.6B
```

2.8.2、参数说明

参数	类型	默认值	说明
`--input`	str	`./dataset/test.jsonl`	输入 JSONL 文件，每行包含 `review_body` 和 `review_title`
`--output`	str	`inference_result.jsonl`	输出 JSONL 文件，每条记录增加 `summary` 字段
`--model_name`	str	`./model_weights/Qwen/Qwen3-0.6B`	基础模型路径
`--lora_path`	str	None	LoRA 权重路径（可选）
`--batch_size`	int	64	一次推理的批量大小
`--sample_size`	int	1280	随机抽样验证条数
`--max_new_tokens`	int	64	生成的最大 token 数
`--min_new_tokens`	int	0	生成的最小 token 数
`--do_sample`	bool	False	是否启用随机采样生成（启用时使用 `--do_sample`）

`--top_p`	float	0.9	top-p 采样的累积概率阈值
`--temperature`	float	0.7	控制生成文本的创造性
`--repetition_penalty`	float	1.1	重复惩罚系数

2.8.3、输出示例

输出文件为 JSONL 格式，每条数据增加 `summary` 字段：

```
{
    "review_body": "物流很快，包装完好。",
    "review_title": "满意的购物体验",
    "summary": "物流快，包装好，购物体验满意。"
}
```

终端输出 ROUGE 结果示例：

```
Sampled 1280 entries BLEU-1: 0.0561, BLEU-2: 0.0258
```

2.9、超参数调整

大家自行调整不同的lora权重、学习率、prompt等

2.10、扩展

- 自行将代码跑一跑，结合ai和debug理解一下代码每一行大概在干啥，不要求写出来，但大体的流程要知道。主要关注一下数据流的变化。
- 你感觉上面的这些评价指标靠谱吗？有更好的办法吗？

三、更换数据集以及优化

3.1、优化

3.1.1、提示词工程优化过程

针对 **Qwen3-0.6B** 小参数模型对指令理解力较弱的问题，进行提示题工程方面的优化：

- **基础版本：**

```
def apply_chat_template(system, user):

    prompt =  f"<|im_start|>system\n{system}<|im_end|>\n<|im_start|>user\n{user}<|im_end|>\n<|i
    return prompt

def generate_review_title_prompt_tmp0(review_body):
    system_prompt = (
        "你将看到一个来自电商平台的商品评论，请根据评论内容，生成简短总结。"
    )
    user_prompt = review_body

    # 使用 apply_chat_template 格式
    return apply_chat_template(system_prompt, user_prompt)
```

- **指令模板规范化**：定义统一的输入格式，明确区分 `Context`（评论内容）、`Task`（生成摘要标题）和 `Constraint`（语言要求、字数限制）。

```
def generate_prompt_structured(review_body):

    system_prompt = (
        "你是一个专业的电商评论总结助手。"
        "你的任务是将评论总结为简短标题。"
    )

    user_prompt = f"""
```

```
    [Context]
    {review_body}

    [Task]
    生成一个总结标题

    [Constraint]
    - 不超过10个字
    - 不要解释
    - 不要标点
    """

    return apply_chat_template(system_prompt, user_prompt)
```

- **Few-shot 推理（少样本引导）**：在 Prompt 中嵌入 2-3 个高质量的“原评论-精炼标题”对照对。这为小模型提供了直观的**输出风格范式**，显著降低了生成内容过长或格式紊乱的概率。

```
def generate_prompt_fewshot(review_body):

    system_prompt = (
        "你是一个电商评论总结专家。"
        "请学习示例并生成相同风格标题。"
    )

    user_prompt = f"""
        示例1:
        评论: 物流很快，包装很好，质量不错
        标题: 物流快质量好

        示例2:
        评论: 颜色和图片不太一样，有点偏暗
        标题: 色差明显

        示例3:
        评论: 穿着很舒服，尺寸合适，很满意
        标题: 舒适合身

        现在请总结:
        - 不超过10个字
        - 不要解释
```

```
        - 不要标点

        评论:
        {review_body}

        标题:
        """

    return apply_chat_template(system_prompt, user_prompt)
```

- **思维链（Chain-of-Thought, CoT）引导**：通过指令引导模型执行“两步走”策略——先要求模型理解，再基于要求模型生成。这种显式推理过程有效补偿了小模型逻辑解析能力的不足。

```
def generate_prompt_fewshot(review_body):

    system_prompt = (
        "你是一个电商评论总结专家。"
        "请学习示例并生成相同风格标题。"
    )

    user_prompt = f"""
        示例1:
        评论: 物流很快，包装很好，质量不错
        标题: 物流快质量好

        示例2:
        评论: 颜色和图片不太一样，有点偏暗
        标题: 色差明显

        示例3:
        评论: 穿着很舒服，尺寸合适，很满意
        标题: 舒适合身

        请一步一步思考:

        步骤1: 理解评论的主要意思
        步骤2: 用10字以内总结为标题
```

```
        现在请总结：
        - 不超过10个字
        - 不要解释
        - 不要标点

        评论：
        {review_body}

        标题：
        """

    return apply_chat_template(system_prompt, user_prompt)
```

3.1.2、推理阶段解码策略调优

为了在“生成多样性”与“逻辑稳定性”之间取得平衡，对解码器参数进行了多次调试：

- **束搜索（Beam Search）**：设置 `num_beams=4` 或 `5`。相比于随机采样，Beam Search 能够维护多个候选路径，最终选择概率累积最高的序列，能有效减少幻觉。
- **核采样与 Top-K 过滤**：

- **Top-K (k=5)**：限制模型仅从概率前 5 的词汇中选择，过滤掉低频长尾词。 - **Top-P (p=0.9)**：动态调整候选集大小，使概率总和达到 90%，在保证流畅度的同时增加了一定的语义灵活性。

- **惩罚系数调优**：

- **Temperature (T=0.7)**：适度调低温度，使概率分布更“陡峭”，让输出更加集中且严谨。 - **Repetition Penalty (1.1-1.2)**：针对 0.6B 模型常见的“复读机”现象，施加重复惩罚，强制模型切换词汇。

```
    generate_config = {
        "max_new_tokens": 32,
        "min_new_tokens": 2,
        "do_sample": do_sample,
        "top_k": 5,
        "top_p": 0.9,
        "temperature": 0.7,
        "repetition_penalty": 1.1
```

```
    }

    with torch.no_grad():
        outputs = model.generate(**inputs, **generate_config)
```

3.1.3、人工评价

（1）评价维度设计（分层打分制，总分 10 分）

人工评估采用三维度评分机制，每个维度对应不同权重，总分为 10 分：

1. **相关性（Relevance, 4分）：** 衡量生成标题是否准确反映原始评论的信息，例如商品质量、物流体验、外观描述或售后反馈等。

- 4分：完整覆盖核心信息 - 2–3分：部分覆盖核心信息 - 0–1分：未反映核心内容或偏离主题

1. **风格一致性（Style Consistency, 3分）：** 衡量生成标题是否符合电商评论标题的表达风格，包括是否具备典型的标题特征，如简洁概括性、关键词导向性和评价导向性，同时与人工标注标题风格保持一致。主要关注以下方面：

- 是否符合电商标题表达习惯（如“物流快质量好”、“色差明显”等） - 是否避免完整句式或冗长描述 - 是否保持总结性而非解释性表达

1. 评分标准：

- 3分：完全符合电商标题风格 - 2分：基本符合，但略有冗余或表达偏离 - 1分：风格明显偏离标题形式 - 0分：完全不符合标题风格

1. **简洁度与流畅度（Fluency, 3分）：** 衡量标题的语言自然程度与可读性，包括语法正确性、表达流畅性以及标题长度控制是否合理。

- 3分：表达自然流畅，无语法错误 - 2分：轻微不自然但不影响理解 - 1分：存在明显语法或表达问题 - 0分：难以理解或表达混乱

（2）盲测流程

为确保评估的客观性与公正性，采用盲测评估流程：

- 从测试集中随机抽取 200 条评论样本；
- 将模型生成标题与人工标注参考标题混合并随机打乱顺序，隐藏生成来源信息，避免评估员（说你评估的或者找的同学）产生偏见；

3.2、更换数据集

3.2.1、项目背景

在数字化办公与智能客服场景中，多轮对话信息往往呈现出碎片化、冗余度高的特点。本项目旨在训练一个能够自动浓缩对话核心要点的摘要算法。通过将冗长的聊天记录转化为简洁的摘要，该算法可服务于**客服工单快速归档以及智能助手长期记忆管理**等实际业务需求，从而显著提升信息流转效率并降低人力审计成本。

3.2.2、数据集介绍

本项目采用了由三星研究院（Samsung R&D Institute Poland）发布的 **SAMSum 对话摘要数据集（kaggle平台可以下载）**。该数据集包含约 1.6 万条模拟真实社交场景的英文对话记录，并由专家进行了高质量的**摘要**标注。

- **数据规模**：包含 14,732 条训练样本、817 条验证样本及 818 条测试样本。
- **数据特性**：对话风格高度拟人化，涵盖了朋友间的即时通讯、日程预约、信息咨询等多种生活场景。
- **数据例子：**

```
{
  "dialogue": "Hannah: Hey, do you have Betty's number?
                  Amanda: Lemme check
                  Hannah: <file_gif>
                  Amanda: Sorry, can't find it.
                  Amanda: Ask Larry
                  Amanda: He called her last time we were at the park together
                  Hannah: I don't know him well
                  Hannah: <file_gif>
                  Amanda: Don't be shy, he's very nice
                  Hannah: If you say so..
                  Hannah: I'd rather you texted him
```

```
                    Amanda: Just text him 🙂
                    Hannah: Urgh.. Alright
                    Hannah: Bye
                    Amanda: Bye bye",
  "summary": "Hannah needs Betty's number but Amanda doesn't have it. She needs to contact Larr
}
```

- `dialogue`：对话内容
- `summary`：对话摘要

3.2.3、代码

下载见：LLM（可下载文档）

此处为动图（内容见飞书原文）

介绍见视频

四、简历怎么写

复制地址：LLM（可下载文档）

项目名称：基于LoRA微调的LLM多语言电商评论摘要自动生成算法

项目描述：针对跨境电商中、英、日三语评论，基于 Qwen3 基座构建自动摘要生成模型，将用户商品评论生成为简短标题，用于客服处理与评价汇总。

技术实现：

- **LoRA 参数高效微调：** 基于 Qwen3 基座模型进行 LoRA 微调，通过调整 LoRA 秩、学习率及作用模块等超参数，实现低成本模型适配与任务优化。
- **提示词工程优化：** 针对小参数模型生成风格不稳定的问题，设计结构化指令模板统一多语言输入格式，引入 Few-shot 示例增强摘要长度控制能力，并结合 CoT 引导模型分步理解评论语

义，提升生成质量与稳定性。

- **推理解码策略调优：** 在推理阶段结合 Beam Search 与 Temperature、Top-K等参数进行解码优化，减少重复生成现象，提高摘要的连贯性与输出稳定性。

项目成果：模型在 **BLEU-1/2** 指标上达到 **0.142**，在更具业务参考价值的**人工评测中获得 7.3 分。**

- 在更具业务参考价值的**人工评测中获得 7.3 分。**具体怎么做的?

见3.1.3

五、面经

0、介绍

1、介绍一下这个项目?

这是一个基于大语言模型的跨境电商多语言评论摘要项目。基于 Qwen3 基座模型，构建了一个摘要生成模型，将中、英、日三语用户评论自动压缩为简短标题，用于客服处理和评价汇总。

在实现上，首先使用 LoRA 对模型进行参数高效微调，使其适应评论摘要任务；然后通过 Prompt 工程引入结构化模板、Few-shot 和 Chain-of-Thought，提高小模型生成的稳定性；最后优化推理解码策略，减少重复生成问题，提升摘要连贯性。最终模型在人工评测中获得了 7.3 / 10 的评分，验证了方法的有效性。

一、数据部分

1、数据集多大？从哪里获得的?

项目数据集构建基于 Amazon Reviews Multi-Language 公开语料，使用中、英、日三种语言。原始数据经过清洗与筛选，最终构建了规模为 **60,000 条的训练集**，以及各 **15,00 条的验证集与测试集。**

在数据处理流程中，项目将原始的评论正文（review_body）与标题（review_title）转化为指令对格式，作为模型微调的输入与输出。这种多语言、大规模的数据组合，确保了模型在处理跨国电商评论摘要任务时，具备良好的语言通用性与语义理解深度，能够准确捕捉不同语种环境下的用户诉求。

二、模型部分

1、LLM的训练有哪些步骤？每一步数据是什么？训练任务是什么？

> 了解即可

LLM 的训练需要经历三个阶段：

预训练 (Pre-training)

- 数据内容：海量的、非结构化的通用文本语料，包括网页、书籍、维基百科、学术论文以及高质量代码库，数据量通常在万亿级 Token 以上。
- 训练任务：**自监督的学习任务，即“下文预测”，**通过最大化给定上文预测下一个词的概率，使模型隐式地习得语法、逻辑、百科知识及简单的推理能力。

有监督微调 (SFT - Supervised Fine-Tuning)

- 数据内容：高质量、人工编写或经过强模型扩充的 “**指令-回复对**”，数据量通常在万级到十万级，SFT 阶段强调数据的质量大于数量。
- 训练任务：**有监督的序列生成任务，**模型学习将输入的指令映射到符合人类习惯的回答格式上，使模型从 “知识库” 向 “对话助手” 转变，激活模型在预训练阶段习得的知识并使模型具备遵循指令的能力。

人类偏好对齐 (RLHF/DPO)

- 数据内容：**偏好排序数据，**针对同一个指令，模型生成多个回答，由人类评价者对结果进行排序。
- 训练任务：**偏好优化任务，**通过强化学习（RLHF）或直接偏好优化（DPO）算法，调整模型输出概率分布，其目的是约束模型的输出，使其不仅满足指令要求，还符合人类的价值观（如安全性、有用性、真实性），同时抑制虚假信息的生成。

2、什么是MoE结构?

MoE 的全称是 **Mixture of Experts**，中文常译为**混合专家模型**。它是一种**稀疏激活**的架构设计。与传统的稠密模型每次推理都激活所有参数不同，MoE 允许模型在大幅提升总参数量（模型容量）的同时，保持较低的计算成本。它通过将 Transformer 块中的全连接层（FFN）替换为一组并行的“专家”层来实现这一点。MoE 结构主要由两个关键部分组成：

- **专家网络（Experts）**：由多个独立的 FFN 层组成。在训练过程中，不同的专家会逐渐演化出对特定知识或任务的处理能力。
- **门控网络（Router/Gating Network）**：这是 MoE 的“大脑”。它负责接收输入 Token 的特征，并实时计算该 Token 应该由哪些专家来处理。

如下图所示，以输入词 “More Parameters” 为例，其处理流程如下：

1. **特征提取**：Token 经过 Self-Attention 和归一化处理后，其 Embedding 进入门控网络。
2. **路由选择**：门控网络计算各专家的权重评分，通常采取 **Top-k 策略**（如 $k=1$ 或 2）。例如，“More” 被路由至 Expert 2，“Parameters” 则被路由至 Expert 1。
3. **结果合并**：只有被选中的专家参与计算，其输出结果按门控权重加权求和后，进入下一层网络。

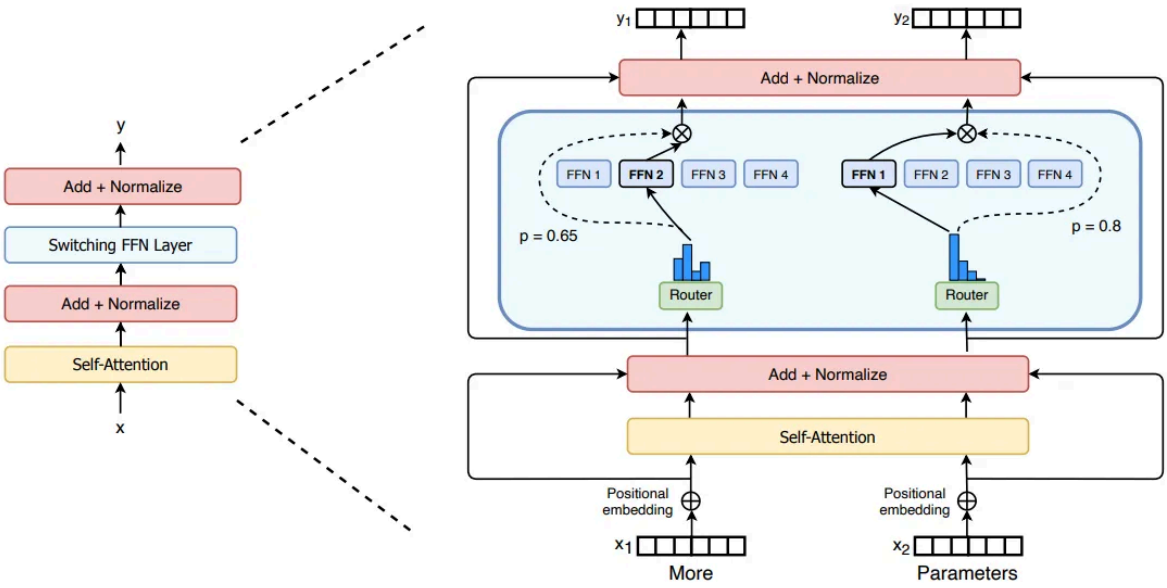


Figure 2: Illustration of a Switch Transformer encoder block. We replace the dense feed forward network (FFN) layer present in the Transformer with a sparse Switch FFN layer (light blue). The layer operates independently on the tokens in the sequence. We diagram two tokens (x_1 = "More" and x_2 = "Parameters" below) being routed (solid lines) across four FFN experts, where the router independently routes each token. The switch FFN layer returns the output of the selected FFN multiplied by the router gate value (dotted-line).

3、decoder only和encoder only有应用任务什么区别？结构上有什么区别？是怎么实现区分的?

> 用于进一步了解模型

（1）应用任务的区别

- **Encoder-only (以 BERT 为代表)**：专注于**语义理解任务。**由于模型可以同时看到上下文的双向信息，它在文本分类、情感分析及语义抽取等需要深层文本理解的场景中表现优异。
- **Decoder-only (以 GPT、Qwen 为代表)**：专注于**内容生成任务。**采用自回归方式，基于已生成的上文逐个预测下一个 Token，适用于对话系统、文章创作、代码生成等生成式场景。

（2）结构上的核心区别：注意力机制

两者在 Transformer 块的层级结构上相似，最本质的区别在于**注意力机制**的交互范围：

- **双向注意力**：

- 应用在 Encoder-only 结构中。 - 每一个 Token 在计算时，都可以“看到”序列中前后所有的 Token 信息。

- **因果掩码注意力**：

- 应用在 Decoder-only 结构中。 - 每一个 Token 只能看到其左侧（已出现）的 Token 信息，无法看到右侧（未来）的信息。

（3）实现区分的技术手段：Mask 矩阵

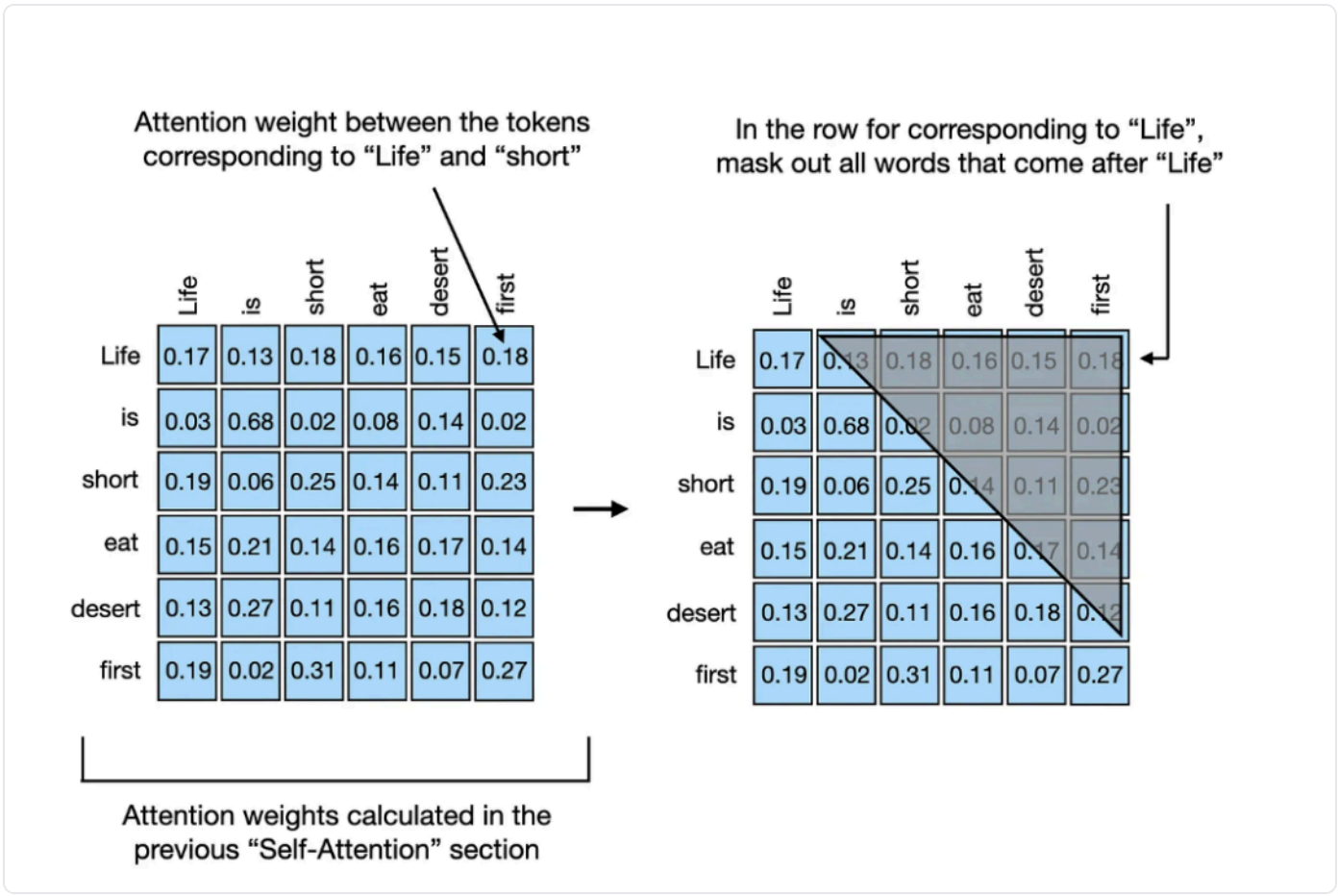
在物理结构上，两者的 Layer 层基本一致，区分交互范围的关键在于**注意力权重计算时的掩码处理（Masking）**：

- **实现原理**：在计算自注意力权重矩阵 QK^T 后，Softmax 操作前，Decoder 会引入一个**上三角掩码矩阵，**引入上三角矩阵之后的如下图所示，灰色部分代表不可交互，即在decoder中，当前的词只能看到之前的，无法看到之后的。
- **数学操作**：将当前位置之后的所有位置的权重值设为 $-\infty$。

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T + M}{\sqrt{d_k}}\right)V$$

其中， M 是掩码矩阵。对于“未来”的 Token 位置，权重被置为负无穷。

- **最终效果**：经过 Softmax 函数处理后，这些负无穷的位置对应的概率值变为 **0**。这意味着在数学层面上，模型预测第 n 个词时，其注意力分布完全无法感知第 $n+1$ 个词及之后的信息，从而实现了“自回归”的特性。



（4）例子：Encoder 与 Decoder 的交互差异

以输入序列 `[中，国，的，首，都，是，哪，里]` 为例，当计算第 6 个 Token **“是”** 的注意力时，两者的交互范围如下：

- BERT (Encoder-only) 交互模式

- **交互范围**：**全量交互。** - **具体表现**：“是” 这个 Token 的查询向量 Q 会与序列中 **全部 8 个 Token** 的键向量 K 和值向量 V 进行计算。 - **特性说明**：由于没有掩码限制，模型具有“全局视野”。在处理“是”时，它不仅知道前文的“中国、首都”，也能感知后文的“哪里”，这使得它能极其精准地理解 Token 在当前语境下的双向语义。

- Qwen/GPT (Decoder-only) 交互模式

- **交互范围**：**单向（因果）交互。** - **具体表现**：“是” 这个 Token 的 Q 仅能与 **前 6 个 Token**（即 `[中，国，的，首，都，是]`）的 K 和 V 进行计算。 - **特性说明**：受**因果掩码（Causal Mask）** 的影响，后文的 `[哪，里]` 在数学上被屏蔽。模型在预测或处理当前词时，只能利用已生成的历史信息，无法“偷看”未来的内容。

4、解码策略有哪些？区别？代码时候怎么实现？

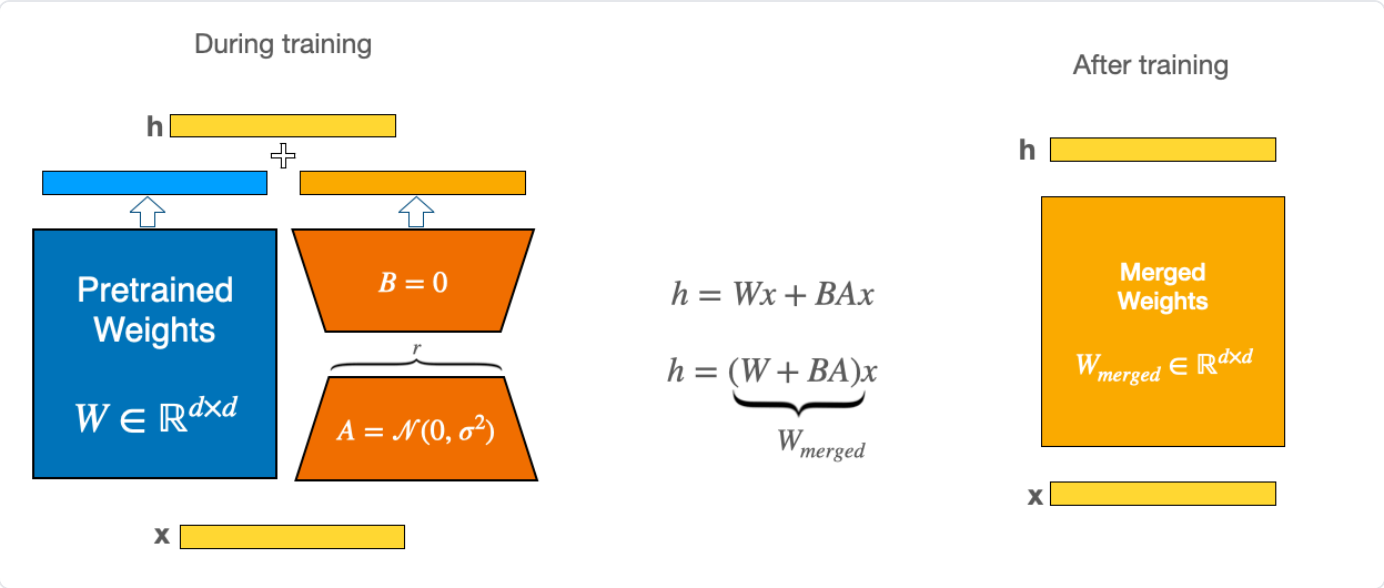
常见的解码策略：

- **贪婪解码**：每一步选择概率最高的词，生成文本连贯但多样性较低。
- **束搜索（Beam Search）**：在贪婪解码基础上，维护多个候选序列，生成更连贯且多样的文本。
- **随机采样（Random Sampling）**：根据词元的概率分布进行采样，每个词被选中的概率与其输出概率成正比。
- **Top-K**：只从概率最高的 K 个词中采样，控制输出多样性并减少重复。
- **Top-P（核心抽样）**：从累计概率达到 P 的词集采样，动态控制候选词数量，提高生成质量。
- **Temperature**：调整概率分布的随机性，低温度生成更确定的文本，高温度生成更丰富的多样化文本。

Hugging Face 的 `generate()` 函数封装了这些方法，我们可以很方便地使用：

```
outputs = model.generate(
    input_ids,
    max_length=50,
    do_sample=True,           # True 表示启用采样，默认为 False，表示贪婪解码
    top_k=50,                 # 从概率最高的 50 个 token 中采样
    top_p=0.9,                # 累积概率达到 0.9 时截断候选集
    temperature=0.8,          # 温度系数
    num_beams=1               # beam数量，>1启用束搜索，=1则不使用
)
```

5、介绍一下lora，以及BA是怎么初始化的？lora在微调完之后，推理的时候，会增加计算量吗？r一般取多少？lora都能作用在哪些fc上？



- **LoRA**：Low-Rank Adaptation 通过优化低秩矩阵的乘积来间接调整神经网络权重矩阵的更新量，冻结原始模型参数的同时，**仅训练新增的低秩分支**，最终可将分支参数与原始权重进行合并，不会增加额外的推理开销：
- **LoRA 原理**：冻结预训练权重 W，并行加一个低秩分解 BA，前向时 $\mathrm{h} = \mathrm{Wx} + \mathrm{BAx}$
- **初始化**：B 初始化为 0，A 用高斯分布初始化，确保训练初期 $\mathrm{BA}{=}0$，不影响原模型
- **推理开销**：推理时可以直接把 BA 加到 W 上变成 $\mathrm{W}' = \mathrm{W} + \mathrm{BA}$，不会增加计算量
- **低秩** r：一般取 8、16、32
- **LoRA 作用位置**：通常作用在 Q/K/V/O 和 FFN 的投影矩阵上。

（1）为什么需要 LoRA?

在大模型微调过程中，全量微调面临的问题：

显存压力:全量微调需要保存模型所有参数的梯度状态。对于拥有数十亿参数的模型，显存需求通常是模型体积的数倍，这使得微调门槛极高。LoRA 通过仅更新低秩矩阵 A 与B ，显著降低了训练时的显存占用。

存储冗余:如果针对不同任务进行全量微调，每个任务都需要存储一份完整的模型副本（动辄数十 GB）。LoRA 采用插件化方案，只需存储数 MB 的适配器参数，不同的任务可以共用同一个底座模型。

（2）什么是Lora?

假设一个预训练模型的原始权重矩阵 W 形状是 5×4（20个参数）：

$$W = \begin{bmatrix} 1.0 & 2.0 & 3.0 & 4.0 \\ 5.0 & 6.0 & 7.0 & 8.0 \\ 9.0 & 10.0 & 11.0 & 12.0 \\ 13.0 & 14.0 & 15.0 & 16.0 \\ 17.0 & 18.0 & 19.0 & 20.0 \end{bmatrix}$$

全量微调需要更新20个参数，假设更新之后的权重矩阵如下：

$$W' = \begin{bmatrix} 1.41 & 2.44 & 3.47 & 4.50 \\ 5.93 & 7.00 & 8.07 & 9.14 \\ 10.45 & 11.56 & 12.67 & 13.78 \\ 14.97 & 16.12 & 17.27 & 18.42 \\ 19.49 & 20.68 & 21.87 & 23.06 \end{bmatrix}$$

微调后的参数 $W' = W + \Delta W$

$$W = \begin{bmatrix} 1.0 & 2.0 & 3.0 & 4.0 \\ 5.0 & 6.0 & 7.0 & 8.0 \\ 9.0 & 10.0 & 11.0 & 12.0 \\ 13.0 & 14.0 & 15.0 & 16.0 \\ 17.0 & 18.0 & 19.0 & 20.0 \end{bmatrix} \quad \Delta W = \begin{bmatrix} 0.41 & 0.44 & 0.47 & 0.50 \\ 0.93 & 1.00 & 1.07 & 1.14 \\ 1.45 & 1.56 & 1.67 & 1.78 \\ 1.97 & 2.12 & 2.27 & 2.42 \\ 2.49 & 2.68 & 2.87 & 3.06 \end{bmatrix}$$

$$W' = W + \Delta W = \begin{bmatrix} 1.41 & 2.44 & 3.47 & 4.50 \\ 5.93 & 7.00 & 8.07 & 9.14 \\ 10.45 & 11.56 & 12.67 & 13.78 \\ 14.97 & 16.12 & 17.27 & 18.42 \\ 19.49 & 20.68 & 21.87 & 23.06 \end{bmatrix}$$

其中增量 ΔW 可通过低秩分解，表示为两个小矩阵的乘积：

$$\Delta W = A \cdot B = \begin{bmatrix} 0.1 & 0.2 \\ 0.3 & 0.4 \\ 0.5 & 0.6 \\ 0.7 & 0.8 \\ 0.9 & 1.0 \end{bmatrix} \cdot \begin{bmatrix} 1.1 & 1.2 & 1.3 & 1.4 \\ 1.5 & 1.6 & 1.7 & 1.8 \end{bmatrix}$$

- **参数对比：** 原本需要训练 **20** 个参数，现在只需训练 $10 + 8 = \mathbf{18}$ 个。

- **实际规模：** 在真实的大模型（如 Llama 3）中，这种压缩比例通常能从 **70亿** 个参数降低到 **几百万** 个，比例达 **0.01%**。

（3）为什么要用“低秩”？

1. 什么是秩（Rank）？

在线性代数中，秩代表一个矩阵中**包含独立信息的维度**。

- 如果一个矩阵的某一行可以由其他行相加得到（线性组合），那么这一行就是“多余”的。
- **低秩**意味着：虽然矩阵看起来很大，但它真正想表达的变化方向其实非常单一。

考虑如下矩阵：

```
[1, 0, 0, 2, 0]
[0, 2, 0, 1, 0]
[1, 1, 1, 1, 1]
[0, 0, 1, 0, 0]
[1, 2, 0, 3, 0]  ← 第5行 = 第1行 + 第2行
```

- 第 5 行是前两行的线性组合，未提供新信息；
- 第 5 列全为 0，也无增量信息。

→ 因此，该矩阵的行秩和列秩均为 **4**。

1. 为什么微调是“低秩”的?

预训练模型已经学习到了海量的通用知识（比如语言结构、基本逻辑）。当我们微调它去写代码或做法律咨询时，模型不需要重塑整个神经系统，只需要在某些“关键特征维度”上做微调。

> **形象类比：**就像给一个顶尖翻译官进行“医学翻译”培训。他不需要从头学说话（W），只需要学习几十个医学专业名词（ΔW）。这些医学名词相对于他掌握的几十万个通用词汇来说，就是“低秩”的。

（4）参数更新

LoRA 的训练本质是训练那两个“小矩阵”：

1. **初始化：** 冻结原始权重 W 。矩阵A用随机高斯分布初始化，矩阵B初始化为**全 0**。

- 注：B 为 0 是为了确保训练开始那一刻， $\Delta W = 0$，模型表现和原版一致，从稳定点出发。

1. **前向传播：** $Output = Wx + (A \cdot B)x$。
2. **计算梯度：** 只计算 Loss 对A和 B的梯度。
3. **优化更新：** 仅更新A和B的参数。
4. **权重合并：** 部署时，可计算 $W_{new} = W + AB$，将知识融入模型，不增加计算量。

（5）超参数如何配置?

在 LoRA 微调中，参数的选择直接影响模型的收敛速度与泛化能力。以下为常用参数的配置建议：

- **秩 r (Rank)：**控制低秩矩阵 A 与 B 的中间维度。通常设定在 8 至 64 之间。对于简单的分类或风格转换任务，r=8 即可满足需求；对于涉及复杂逻辑推理的任务，建议增加至 16 或 32。
- **缩放因子 α：** 用于调整 LoRA 权重对原始模型的影响权重。通常将其设定为r的两倍。该参数起到了归一化的作用，有助于在不同r值下保持训练过程的稳定性。
- **目标模块:** 指定应用 LoRA 的模型层。建议覆盖模型中所有的线性层（包括 FFN），这有助于提升微调效果的稳定性与上限。

Dropout 比例:设定在 0.05 至 0.1 之间。通过在训练中随机失活部分神经元，增强模型的鲁棒性，防止模型过度拟合训练数据。

（6）调优技巧与策略

从小规模参数起步:多数任务建议从 $r = 8$ 或 $r = 16$ 开始尝试。根据验证集的性能表现，再决定是否通过增大 r 来提升模型容量。

基于数据量选择配置

- **小数据集（< 5,000 条）**： 建议采用 r=8 并覆盖全模块，以极低参数量获得最高性价比。
- **大数据集（> 50,000 条）**： 可尝试将 r 提升至 32 或 64，以捕捉数据中更复杂的特征分布。

6、介绍一下在prompt阶段的增强的方法?

Prompt 阶段的常见增强方法：

- **CoT (思维链)**：通过示例引导模型输出“推理步骤”
- **零样本CoT**：插入“让我们一步步思考”引导推理
- **Few-shot**：提供 3-5 个标准的“输入-输出”示例
- **Self-Correction（自洽性）**：生成多个回答并进行投票

三、损失函数

1、使用了什么损失函数?

生成任务本质上属于多分类问题，因此采用交叉熵损失函数作为训练目标函数，以衡量模型预测分布与真实分布之间的差异。

四、优化器

2、使用了什么优化器?

在优化算法方面，使用 Adam 优化器对模型参数进行更新，以获得较好的收敛性能和训练稳定性。

五、训练超参数

1、使用模型什么型号？为什么?

本项目选用 Qwen3-0.6B 作为基础模型，主要考虑在有限算力资源条件下兼顾模型性能与训练成本。

2、LLM在微调的时候，怎么看显存占用多少？使用什么卡微调的？r给的多少？都尝试了哪些超参数？哪些超参数对结果影响最大?

在微调过程中，使用 **NVIDIA RTX 4090 (24GB)** 单卡进行实验。通过 `nvidia-smi` 监控显示，在 **Batch Size** 为 4 的配置下，显存占用约为 **14GB**。LoRA 微调的 **r** 尝试了 4、8、16 三个量级，实验发现其对任务指标提升的边际效应递减，综合资源消耗与性能表现，最终选定 **r=8**。

在超参数调优方面，调整了学习率、Batch Size及温度系数等指标。实验结论显示，**学习率**与 **r** 对结果影响最为显著：学习率直接决定训练的稳定性与收敛速度，而 r 的取值则在模型拟合能力与显存开销之间起平衡作用。

3、训练超参数是多少？训练了多久？

采用了 **Qwen3-0.6B** 作为底座模型，并基于 **LoRA**技术进行高效微调。超参数配置如下：

1. **LoRA 结构参数**

将秩（Rank）设定为 r=8，缩放因子 `lora_alpha = 32` 表示将低秩更新项按 $\frac{\alpha}{r} = \frac{32}{8} = 4$ 倍比例缩放，并应用了 0.1 的 Dropout 以增强模型泛化能力。为了保证微调效果的稳定性，LoRA 权重覆盖了模型中所有的线性层，包括注意力机制中的 q, k, v, o_proj 以及 MLP 层的 $gate, up, down_proj$。

1. **训练训练超参数**

训练过程设定为 3 个 Epoch，总计样本量为 60,000 条。学习率配置为 1×10^{-4}。在批处理策略上，单个设备的 Batch Size 设为 4，并配合 4 步梯度累积。

1. **硬件开销与耗时**

实验在单张 **RTX 4090 (24GB)** 显卡上运行。得益于 LoRA 的低参数量特性与 Qwen3-0.6B 较小的参数规模，整体训练任务在约 **2 小时** 完成。

1. **推理采样配置**

推理阶段采用了 Top-p 采样策略，阈值设为 0.9，并将 Temperature 控制在 0.7，以平衡生成文本的确定性与创造性。

4、Prompt怎么给的？

也可以尝试其他的prompt

```
def generate_prompt_fewshot(review_body):

    system_prompt = (
        "你是一个电商评论总结专家。"
        "请学习示例并生成相同风格标题。"
    )

    user_prompt = f"""
        示例1:
        评论: 物流很快，包装很好，质量不错
        标题: 物流快质量好

        示例2:
        评论: 颜色和图片不太一样，有点偏暗
        标题: 色差明显

        示例3:
        评论: 穿着很舒服，尺寸合适，很满意
        标题: 舒适合身

        请一步一步思考:

        步骤1: 理解评论的主要意思
        步骤2: 用10字以内总结为标题

        现在请总结:
        - 不超过10个字
        - 不要解释
        - 不要标点

        评论:
        {review_body}

        标题:
        """

    return apply_chat_template(system_prompt, user_prompt)
```

六、评价指标

1、介绍一下BLEU-2评价指标?

> 重要

BLEU-2 计算生成文本和参考文本的 2-gram 重合度，即统计生成文本中连续 2 个词的组合有多少出现在参考文本中，其数学公式为：

$$\text{BLEU-2} = \text{BP} \cdot \exp\left(\sum_{n=1}^{2} w_n \log p_n\right)$$

- p_n：n-gram 精度（候选译文的 n-gram 在参考译文中出现的比例）
- BP：短句惩罚因子，惩罚过短输出
- w_n：n-gram 权重（BLEU-2 仅用 1-gram + 2-gram，$w_n = \frac{1}{2}$）

2、在更具业务参考价值的人工评测中获得 7.3 分。具体怎么做的?

见3.1.3人工评价指标。

七、优化

1、你认为最难的？或者有遇到什么问题？怎么解决的?

在实验过程中，首要**硬件显存限制**。为在有限的计算资源下完成全流程验证，选用了参数规模较小的 **0.6B 模型**作为基座。虽然小参数模型降低了算力门槛，但也带来了生成风格不稳定等问题。

针对**生成风格差异大**的问题，采取的优化策略：一方面，通过**提示词工程** 强化对输出长度和语气的约束，确保生成指令的规范化；另一方面，对 **LoRA Rank** 及学习率等关键超参数进行实验，动态调整可训练参数的规模。这种方法有效抑制了小模型在微调中容易出现的过拟合现象，最终在显存消耗与生成质量之间取得了较好的平衡。

2、你认为这种评价指标靠谱吗？你认为真实的评价指标应该是怎么样的?

关于评价指标，我认为传统的 **BLEU** 等算法存在天然缺陷。其本质是基于字面的匹配，无法感知深层语义。例如，“北京是中国的首都”与“中国的首都是北京”在语义上完全一致，但因语序和词偏好差异，BLEU 得分往往偏低；相反，一些字面重复度高但逻辑事实错误的句子，却可能获得虚高评分。

我认为真实可靠的评价体系应当是**多维度分层设计**的：

1. **语义相似度**：利用 Embedding 向量计算余弦相似度，从向量空间捕捉语义对齐。
2. **LLM-as-a-Judge**：引入 GPT-4 等强语言模型作为自动化评委，从逻辑性、事实正确性及指令遵循度进行多维度打分。
3. **人工评估**：针对最终交付场景，从相关性、风格一致性以流畅度进行人工抽样校验。

QA

- 你对LLM的哪些部分进行微调了（或者说LoRA作用在哪些linear上面?）

- 见红色，7种类。

```
# 配置LoRA适配器
    if use_lora:
        lora_config = LoraConfig(
            task_type=TaskType.CAUSAL_LM,
            target_modules=[  # 覆盖关键注意力模块
                "q_proj", "k_proj", "v_proj", "o_proj",
                "gate_proj", "up_proj", "down_proj"
            ],
            inference_mode=False,
            r=8,                    # LoRA秩（平衡效果与效率）
            lora_alpha=32,           # 缩放因子（一般=2r 或 4r）
            lora_dropout=0.1,         # 正则化比率
        )
        model = get_peft_model(model, lora_config)
```

- transformer有哪些优缺点？后续有哪些改进？

- 优点：1、解决长距离依赖（比如 lstm), 2、可以并行化； - 缺点： - 1、计算量大 - 解决1：分组查询注意力（Grouped Query Attention, GQA）是在 MHA 基础上的一种优化形式：**多个 Query 头共享同一组 Key 和 Value 投影。**这样可以显著减少 Key/Value 的参数量和缓存开销，提高推理效率，尤其适用于大模型推理加速。GQA 在保持较好建模能力的同时，大幅降低了计算和内存成本，是当前大模型中常用的高效注意力机制。 - 解决2: 稀疏注意力：选择部分的token进行交互。https://zhuanlan.zhihu.com/p/1957032283270812718 - 解决3:MoE，参数大，但是激活小。 - 2、位置编码 - 绝对位置编码和相对位置编码，训练和推理可能不一致，现在LLM位置编码基本repo。

- 强化学习了解多少？强化学习是一种什么学习范式

- 强化学习没有标准标签，但有“奖惩”机制，模型通过不断试错，学习如何做出能获得最多奖励的决策。 - DPO GRPO

- lora微调中秩对学习率的影响?

- 秩越大 → 参数更多 → 更新更多 → 通常需要更小学习率 秩越小 → 参数受限 → 更新更“保守” → 可以用更大学习率 - 为了保证这种不同秩的影响，我们lora微调时候，有一个缩放因子 α： - 用于调整 LoRA 权重对原始模型的影响权重。通常将其设定为r的两倍。该参数起到了归一化的作用，有助于在不同r值下保持训练过程的稳定性。

- 温度对重复生成的作用?

- **温度越低 → 越容易重复 温度越高 → 越不容易重复（但可能变乱）**

- LoRA本质上为什么会有效?

- 因为作用的矩阵，本身的秩比较低。比如q_proj 是一个768*768的矩阵，但是秩比较低。

- 超参数怎么给的或者怎么调整出来的?

- Batch size：在不超过显存的情况下，尽可能大，具体占用了多少显存通过 `nvidia-smi` 查看，如果还没占满，就继续大，如果报错，超显存了，就调小，一般都是2的倍数。 - 学习率和epoch：这两个超参数是相关的，一般学习率小的话，训练的epoch就会多一些； - 学习率：微调的话，一般给1e-4或者1e-5（具体哪个好，需要跑实验看看） - epoch：开始给的尽可能大，发现loss收敛或者模型在验证集上的性能不在提升，就可以停止了。 - 优化器类别： - 这种一般adam最好，但也不是绝对的，其他的可以跑一下试试， - LoRA相关的： - r大小：一般给8，在计算量允许的情况下，也可以给16、32等试试，效果就提升的话，可以给更大的。 - 作用的模块：全部作

用：**"q_proj", "k_proj", "v_proj", "o_proj", "gate_proj", "up_proj", "down_proj"** - dropout比例：一般0.1，也可以调整看看，不跑实验，不知道具体多少好，需要跑实验看。